

PAR KALONJI
KALALA

Apprentissage par renforcement (Reinforcement Learning)

Introduction

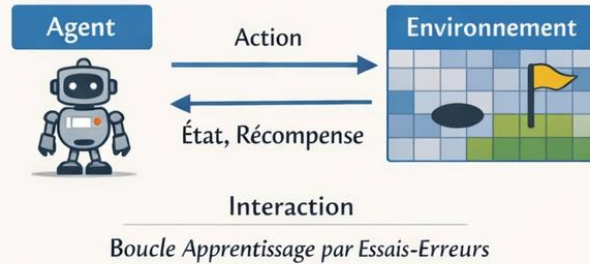
- ❑ **Le Reinforcement Learning** est un paradigme d'apprentissage automatique dans lequel un **agent autonome** apprend à prendre des décisions en **interagissant avec un environnement**.
- ❑ **Contrairement à l'apprentissage supervisé, il n'y a pas de données étiquetées** : l'agent apprend par **essais-erreurs**, guidé par des **récompenses**.
- ❑ L'objectif de l'agent est de **maximiser la récompense cumulée** à long terme en **apprenant une stratégie optimale, appelée politique**.

Introduction

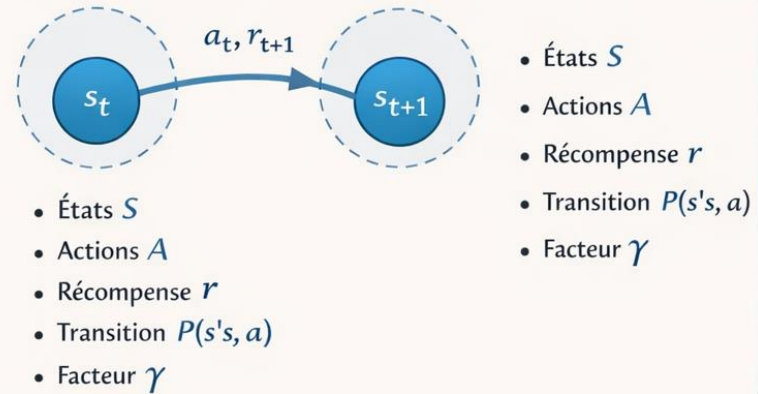
- ❑ **Le Reinforcement Learning (apprentissage par renforcement)** est un domaine de l'apprentissage automatique dans lequel un **agent autonome** apprend à prendre des décisions **par interaction avec un environnement**, dans le but de **maximiser une récompense cumulative**.
- ❑ Contrairement à :
 - ❑ l'apprentissage supervisé (données étiquetées),
 - ❑ l'apprentissage non supervisé (pas d'étiquettes),
- ❑ le RL repose sur **l'expérience** et le **feedback** de l'environnement.

Introduction au Reinforcement Learning (RL)

1. Modèle Agent—Environnement



2. Processus de Décision de Markov (MDP)



3. Algorithme Q-Learning

Q-table

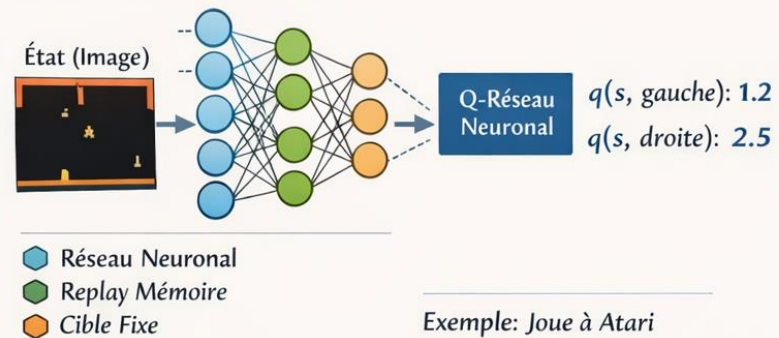
1			1
1			1
1	1	1	

Exemple: Frozen Lake



$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

4. Deep Q-Network (DQN)



Problème d'agent autonome

- Un problème de RL est généralement modélisé comme un **agent autonome** qui :
 - **perçoit son environnement,**
 - **agit,**
 - **observe les conséquences de ses actions,**
 - **s'améliore au fil du temps.**

- **Boucle d'interaction fondamentale :** Agent $\leftrightarrow$ Environnement

- **À chaque étape :**
 1. l'agent observe l'état,
 2. il choisit une action,
 3. l'environnement réagit (l'environnement évolue vers un nouvel état)
 4. une récompense est retournée,
 5. un nouvel état est observé.

Concepts fondamentaux : États (States)

□ Un **état** représente la situation courante de l'environnement telle qu'elle est perçue par l'agent.

□ **Exemples :**

- position d'un robot,
- configuration d'un jeu,
- situation financière d'un marché.

□ On note généralement l'état par :

$$s_t \in \mathcal{S}$$

Concepts fondamentaux : Actions

❑ Une **action** est une décision prise par l'agent pour influencer l'environnement.

❑ **Exemples :**

- ❑ avancer / reculer,
- ❑ tourner à gauche / droite,
- ❑ acheter / vendre.
- ❑ déplacer un robot (avancer, tourner),
- ❑ acheter/vendre dans un marché financier,
- ❑ jouer un coup dans un jeu (gauche, droite, tirer).

❑ **Notation :**

$$a_t \in \mathcal{A}$$

Concepts fondamentaux : Récompenses (Rewards)

- ❑ La **récompense** est un signal numérique fourni par l'environnement pour évaluer l'action de l'agent.
 - ❑ positive : action souhaitable,
 - ❑ négative : action indésirable.

- ❑ **Notation :**

$$r_t = R(s_t, a_t)$$

- ❑ **Objectif de l'agent : maximiser la somme des récompenses futures.**

Concepts fondamentaux : Politique (Policy)

□ La **politique** définit le comportement de l'agent : elle associe à chaque état une action.

$$\pi(a | s)$$

- **politique déterministe** : une action par état,
- **politique stochastique** : une distribution de probabilité.

Fonction de valeur

□ La **fonction de valeur** mesure la qualité d'un état ou d'une action.

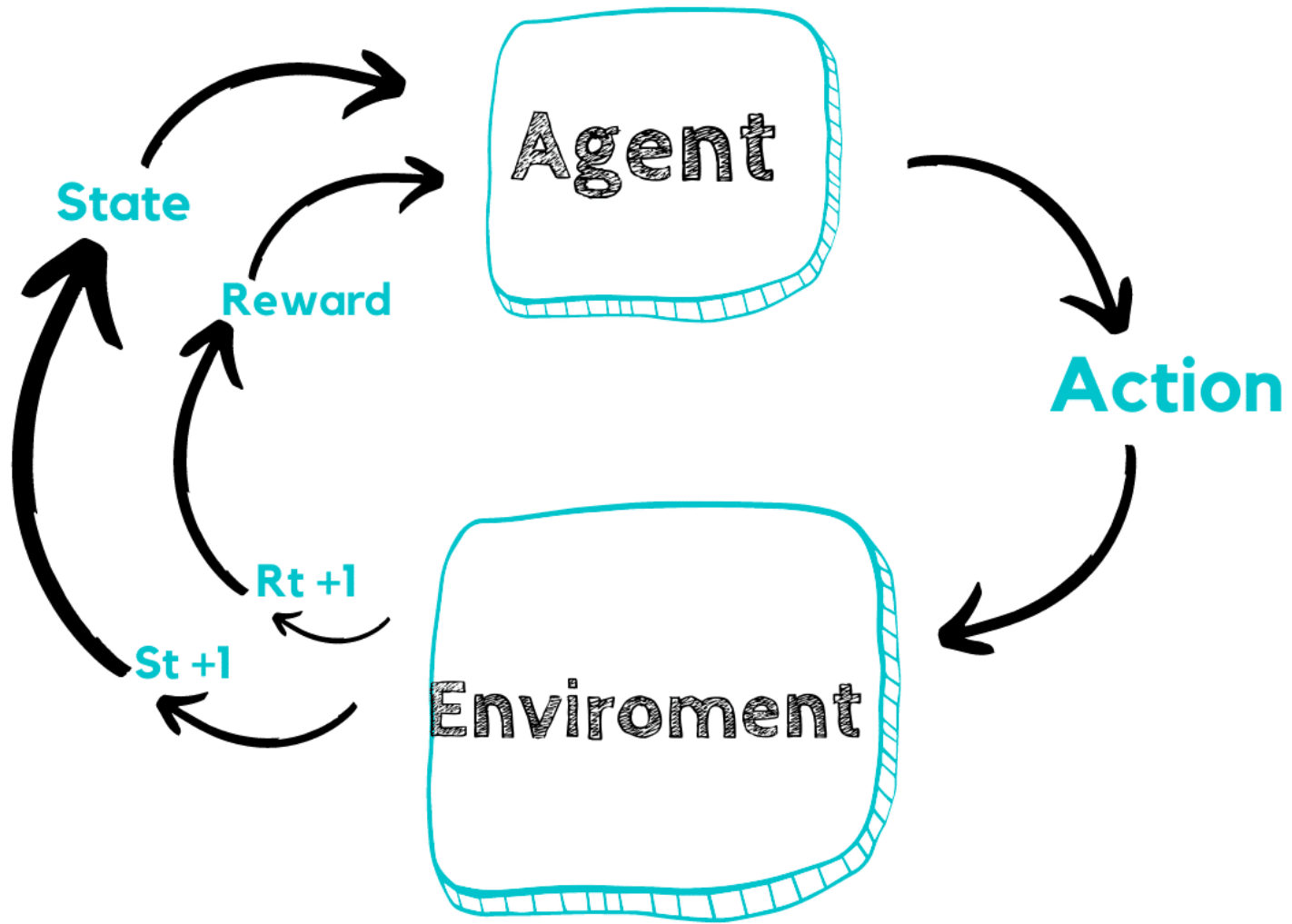
□ **Valeur d'un état :**

$$V^{\pi}(s)$$

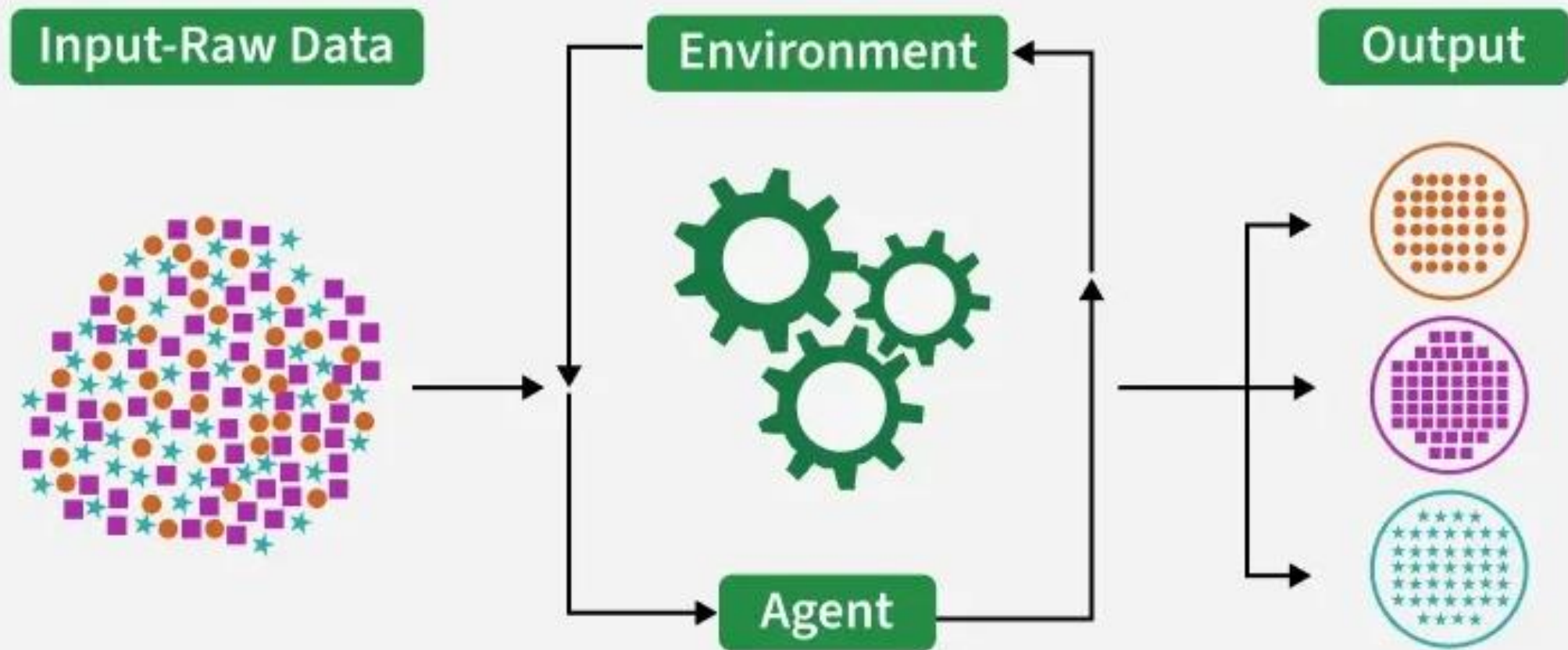
□ **Valeur état-action :**

$$Q^{\pi}(s, a)$$

□ Elle représente la **récompense cumulée espérée** à long terme.



Reinforcement Learning in ML



REINFORCEMENT LEARNING

Reinforcement learning is a machine learning paradigm that focuses on how agents learn to interact with an environment to maximize cumulative rewards.

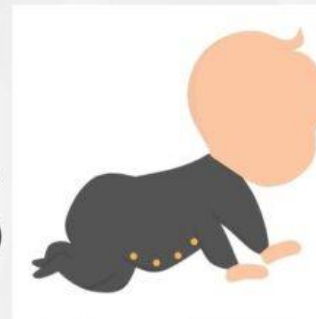


Baby (Agent)



Sitting

State (Action)



Crawling

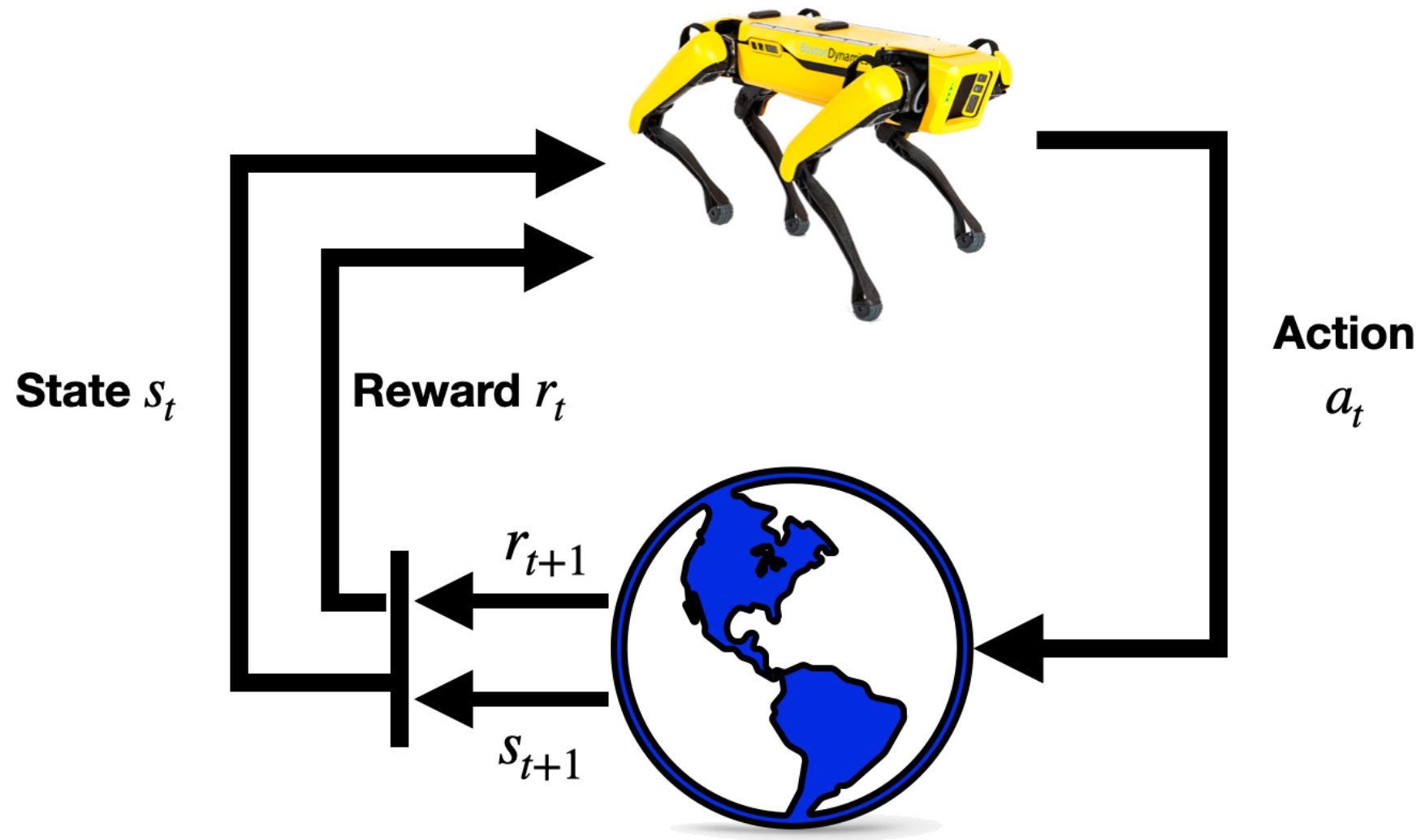
Reward



Feeder

Algorithms and Approaches in Reinforcement Learning

- Q-learning
- Deep Q-networks (DQN)
- Policy Gradients Methods
- Proximal Policy Optimization (PPO)



Dog (Agent)



Sitting

State (Action)



Dog Training Environment



Catch the Boll

Reward



Simple code : environnement jeux :

<https://github.com/hk027/Lacite/blob/main/DeepLearning/nonDLenvironment.ipynb>

Apprentissage par renforcement (Reinforcement Learning)

Introduction

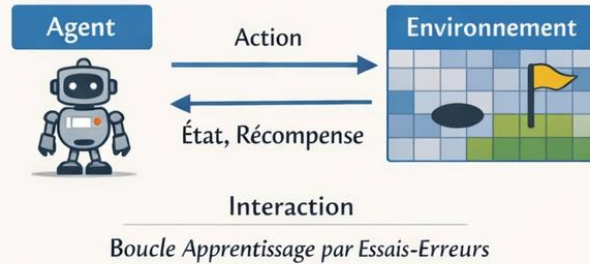
- ❑ **Le Reinforcement Learning** est un paradigme d'apprentissage automatique dans lequel un **agent autonome** apprend à prendre des décisions en **interagissant avec un environnement**.
- ❑ **Contrairement à l'apprentissage supervisé, il n'y a pas de données étiquetées** : l'agent apprend par **essais-erreurs**, guidé par des **récompenses**.
- ❑ L'objectif de l'agent est de **maximiser la récompense cumulée** à long terme en **apprenant une stratégie optimale, appelée politique**.

Introduction

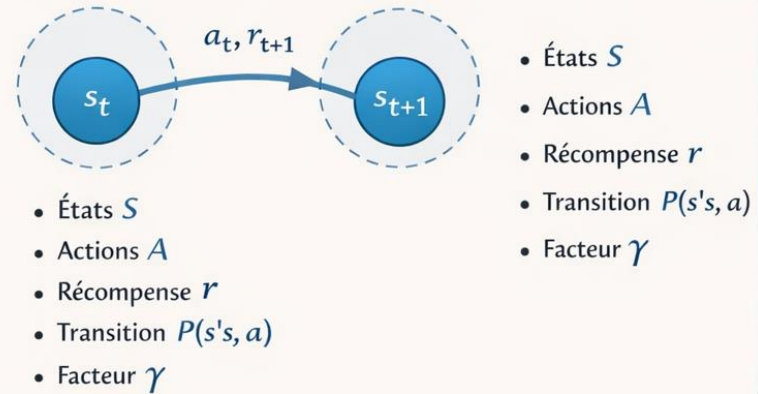
- ❑ **Le Reinforcement Learning (apprentissage par renforcement)** est un domaine de l'apprentissage automatique dans lequel un **agent autonome** apprend à prendre des décisions **par interaction avec un environnement**, dans le but de **maximiser une récompense cumulative**.
- ❑ Contrairement à :
 - ❑ l'apprentissage supervisé (données étiquetées),
 - ❑ l'apprentissage non supervisé (pas d'étiquettes),
- ❑ le RL repose sur **l'expérience** et le **feedback** de l'environnement.

Introduction au Reinforcement Learning (RL)

1. Modèle Agent—Environnement



2. Processus de Décision de Markov (MDP)



3. Algorithme Q-Learning

Q-table

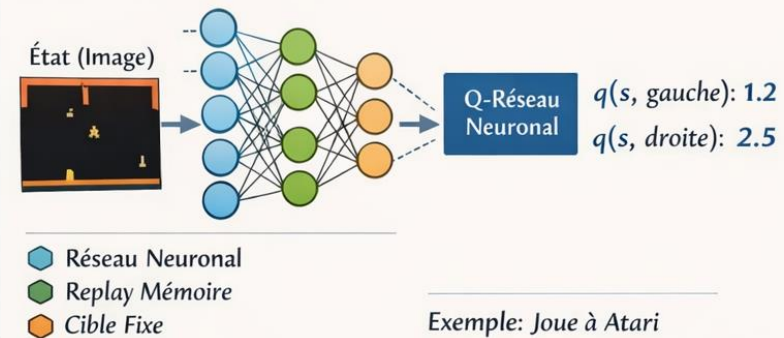
1			1
1			1
1	1	1	

Exemple: Frozen Lake



$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

4. Deep Q-Network (DQN)



Problème d'agent autonome

- Un problème de RL est généralement modélisé comme un **agent autonome** qui :
 - **perçoit son environnement,**
 - **agit,**
 - **observe les conséquences de ses actions,**
 - **s'améliore au fil du temps.**

- **Boucle d'interaction fondamentale :** Agent $\leftrightarrow$ Environnement

- **À chaque étape :**
 1. l'agent observe l'état,
 2. il choisit une action,
 3. l'environnement réagit (l'environnement évolue vers un nouvel état)
 4. une récompense est retournée,
 5. un nouvel état est observé.

Concepts fondamentaux : États (States)

□ Un **état** représente la situation courante de l'environnement telle qu'elle est perçue par l'agent.

□ **Exemples :**

- position d'un robot,
- configuration d'un jeu,
- situation financière d'un marché.

□ On note généralement l'état par :

$$s_t \in \mathcal{S}$$

Concepts fondamentaux : Actions

❑ Une **action** est une décision prise par l'agent pour influencer l'environnement.

❑ **Exemples :**

- ❑ avancer / reculer,
- ❑ tourner à gauche / droite,
- ❑ acheter / vendre.
- ❑ déplacer un robot (avancer, tourner),
- ❑ acheter/vendre dans un marché financier,
- ❑ jouer un coup dans un jeu (gauche, droite, tirer).

❑ **Notation :**

$$a_t \in \mathcal{A}$$

Concepts fondamentaux : Récompenses (Rewards)

- ❑ La **récompense** est un signal numérique fourni par l'environnement pour évaluer l'action de l'agent.
 - ❑ positive : action souhaitable,
 - ❑ négative : action indésirable.

- ❑ **Notation :**

$$r_t = R(s_t, a_t)$$

- ❑ **Objectif de l'agent : maximiser la somme des récompenses futures.**

Concepts fondamentaux : Politique (Policy)

□ La **politique** définit le comportement de l'agent : elle associe à chaque état une action.

$$\pi(a | s)$$

- **politique déterministe** : une action par état,
- **politique stochastique** : une distribution de probabilité.

Fonction de valeur

□ La **fonction de valeur** mesure la qualité d'un état ou d'une action.

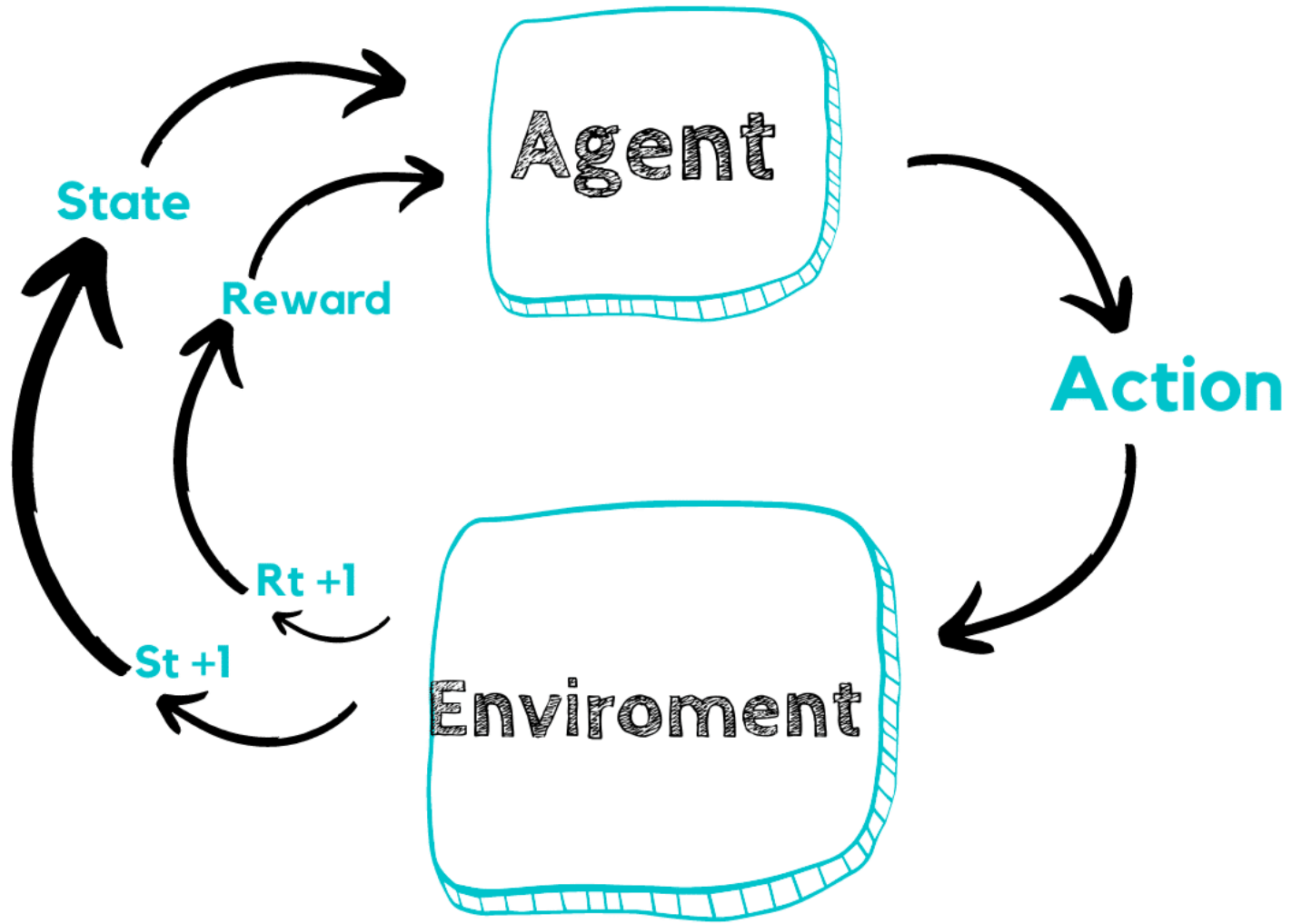
□ **Valeur d'un état (state):**

$$V^{\pi}(s)$$

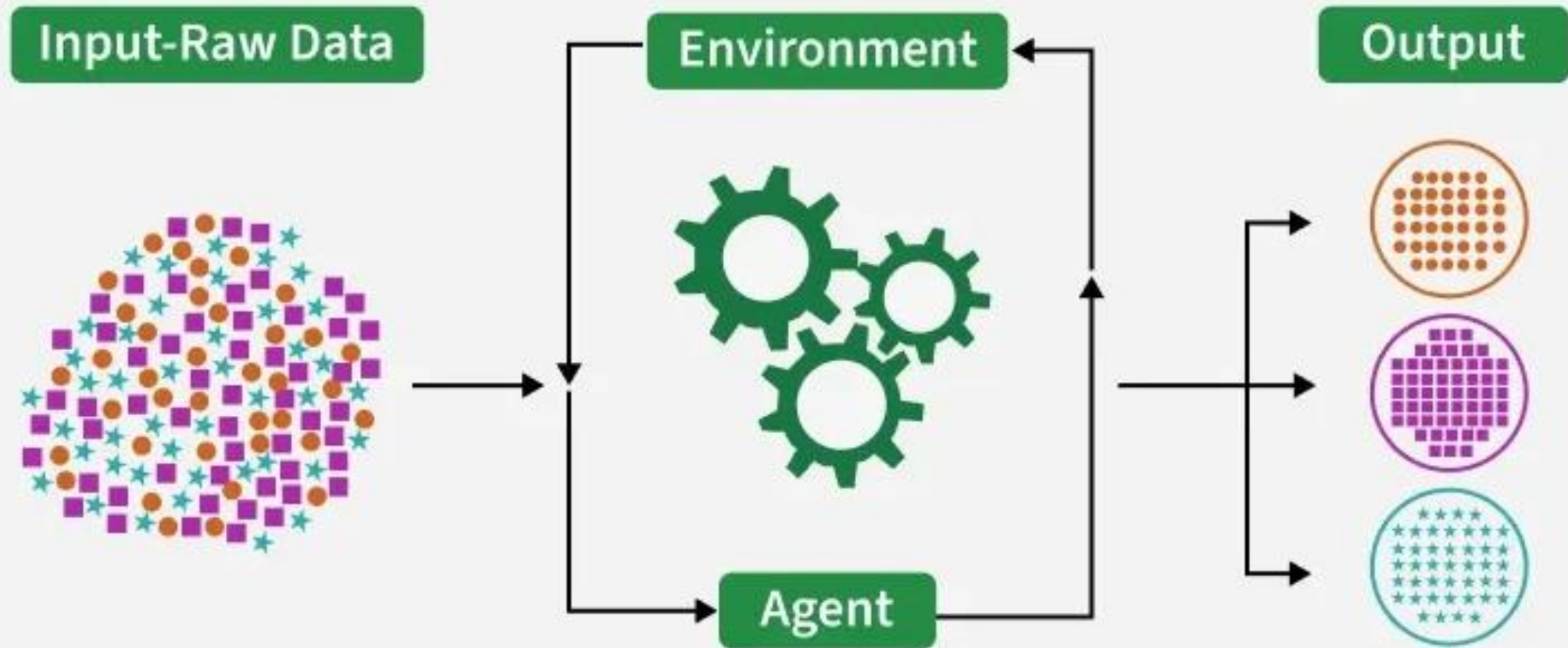
□ **Valeur état-action :**

$$Q^{\pi}(s, a)$$

□ Elle représente la **récompense cumulée espérée** à long terme.



Reinforcement Learning in ML



REINFORCEMENT LEARNING

Reinforcement learning is a machine learning paradigm that focuses on how agents learn to interact with an environment to maximize cumulative rewards.

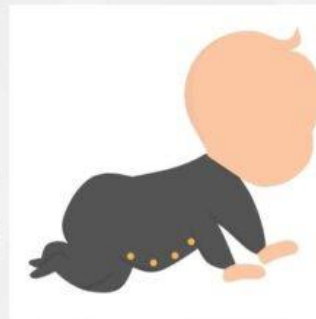


Baby (Agent)



Sitting

→
State (Action)



Crawling

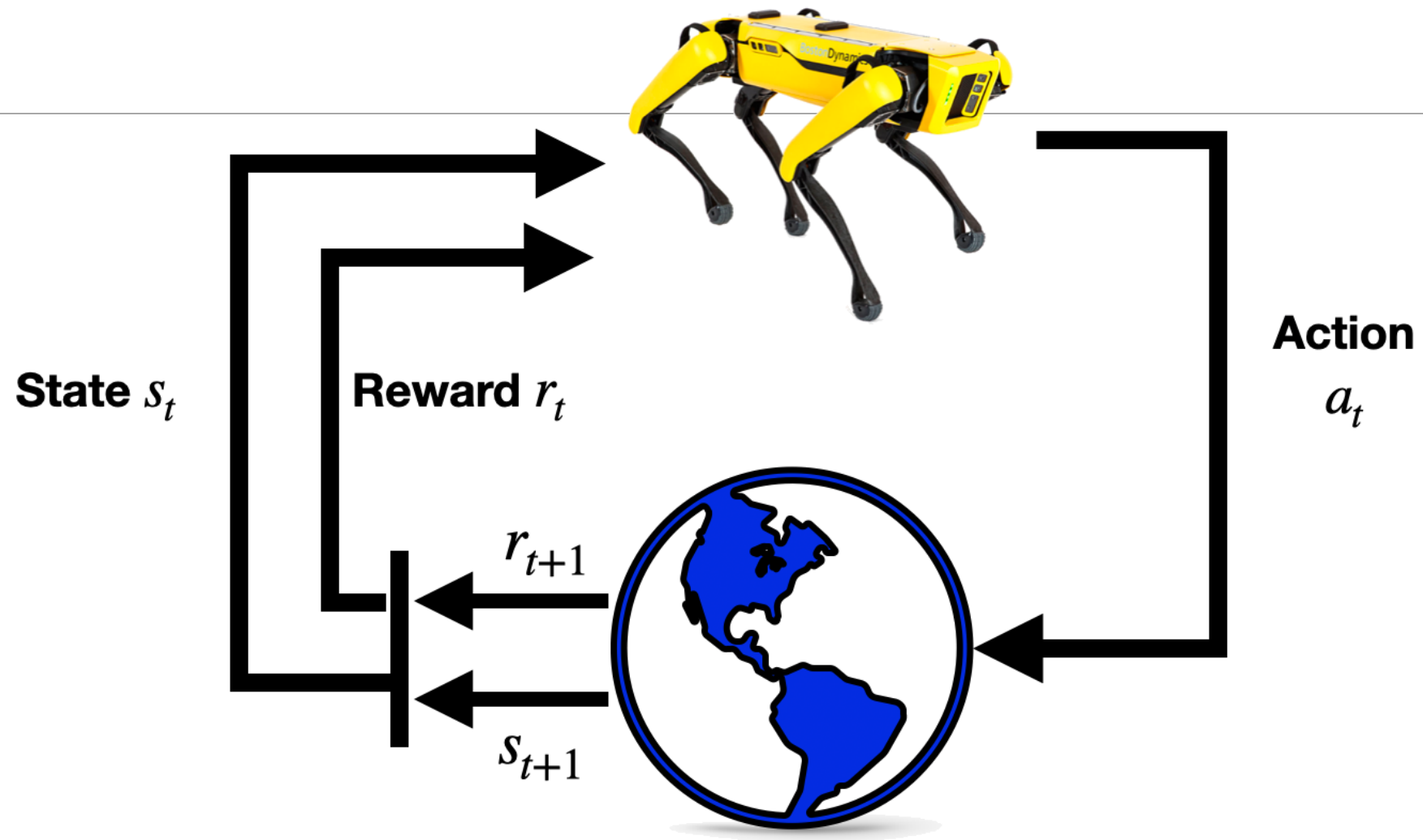
←
Reward



Feeder

Algorithms and Approaches in Reinforcement Learning

- Q-learning
- Deep Q-networks (DQN)
- Policy Gradients Methods
- Proximal Policy Optimization (PPO)



Dog (Agent)



Sitting

State (Action)



Dog Training Environment



Catch the Boll

Reward



Simple code : environnement jeux :

<https://github.com/hk027/Lacite/blob/main/DeepLearning/nonDLenvironment.ipynb>

- ❑ PRINCIPALES
BIBLIOTHÈQUES
UTILISÉES POUR :
- ❑ CRÉER DES
ENVIRONNEMENTS
- ❑ CONSTRUIRE DES
RÉSEAUX DE
NEURONES
- ❑ IMPLÉMENTER DES
ALGORITHMES RL
- ❑ SUIVRE
L'APPRENTISSAGE

Bibliothèques de Deep Reinforcement Learning en Python

Gymnasium

Objectif : Simulation d'environnements RL

- ❑ Installation : pip install gymnasium
- ❑ Avantages : Interface standardisée, facile à utiliser
- ❑ Grand nombre d'environnements
- ❑ Inconvénients : Environnements simples
- ❑ Réalisme limité pour simulations complexes

PyTorch

Objectif : Construire des réseaux de neurones

- Installation : `pip install torch`
- Avantages : Graphes dynamiques
- Débogage facile
- Très utilisé en recherche
- Inconvénients : Implémentation manuelle RL

Stable-Baselines3

Objectif : Algorithmes RL prêts à l'emploi

- Installation : `pip install stable-baselines3`
 - Avantages : API simple
 - Implémentations fiables
 - Expérimentation rapide
 - Inconvénients : Personnalisation limitée
-
- Stable Baselines3 (SB3) est un ensemble d'implémentations fiables d'algorithmes d'apprentissage par renforcement dans PyTorch.

RLlib

Objectif : RL distribué

- Installation : `pip install ray[rllib]`
- Avantages : Multi-agents
- Apprentissage distribué
- Prêt pour production
- Inconvénients : Configuration complexe

TF-Agents

Objectif : RL basé sur TensorFlow

- Installation : `pip install tf-agents`
- Avantages : Architecture modulaire
- Bonne intégration TensorFlow
- Inconvénients : Moins accessible débutants

TensorBoard

Objectif : Visualisation

- Installation : `pip install tensorboard`
- Avantages : Visualise récompenses
- Suivi entraînement
- Inconvénients : Configuration requise

Notions de MDP (A Markov Decision Process

Processus de Décision de Markov (MDP)

- ❑ Dans l'apprentissage par renforcement, **un agent interagit avec un environnement en prenant des décisions séquentielles afin de maximiser une récompense cumulative.**
- ❑ Ce type de problème est modélisé mathématiquement par un **Processus de Décision de Markov (Markov Decision Process – MDP).**
- ❑ Un MDP est défini par un ensemble de composantes qui décrivent la dynamique de l'environnement et la manière dont l'agent peut interagir avec celui-ci.

Processus de Décision de Markov (MDP)

La majorité des problèmes de RL sont formalisés comme un processus de décision markovien (MDP) :

$$\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$$

où :

- $\mathcal{S}$: ensemble des états,
- $\mathcal{A}$: ensemble des actions,
- P : probabilités de transition,
- R : fonction de récompense,
- $\gamma \in [0, 1]$: facteur d'actualisation (Facteur de discount).

Le facteur γ permet de donner plus ou moins d'importance au futur.

Processus de Décision de Markov (MDP)

États (States)

- Un **état** représente une situation dans laquelle l'agent peut se trouver à un instant donné.
- L'ensemble de tous les états possibles est généralement noté S .
- À chaque étape temporelle t , l'agent observe l'état courant s_t , qui contient toutes les informations nécessaires pour prendre une décision.

Actions (Actions)

- À partir d'un état donné, l'agent peut choisir une action parmi un ensemble d'actions possibles, noté A . L'action sélectionnée à l'instant t est généralement notée a_t .
- Le choix de cette action influence la transition vers l'état suivant.

Processus de Décision de Markov (MDP)

Probabilité de Transition (Transition Probability)

□ **Lorsqu'un agent effectue une action dans un état donné, l'environnement passe à un nouvel état selon une certaine probabilité.** Cette dynamique est modélisée par la probabilité de transition :

$$P(s' | s, a)$$

qui représente la probabilité de passer à l'état s' lorsque l'agent exécute l'action a dans l'état s

Processus de Décision de Markov (MDP)

Probabilité de Transition (Transition Probability) : Interpretation

Cette probabilité modélise l'incertitude de l'environnement.

❑ Si l'environnement est déterministe :

❑ Une action mène toujours au même état.

❑ **Exemple :** $P(s'|s, a) = 1$

❑ Si l'environnement est stochastique :

❑ Une même action peut mener à plusieurs états possibles.

❑ **Exemple :**

❑ 70% de chance d'aller à droite

❑ 30% de chance de rester sur place

Donc ici :

❑ $P(droite|s, a) = 0.7$

❑ $P(rester|s, a) = 0.3$

MDP : Illustration de la probabilité de transition dans un environnement stochastique

Situation initiale

- ❑ État actuel : (0,0)
- ❑ Action : Aller à droite
- ❑ Le sol est glissant

Grille avec Robot et Transitions possibles

Robot (0,0) 0.1	(0,1) 0.8	(0,2)
(1,0) 0.1	(1,1)	(1,2)
(2,0)	(2,1)	(2,2)

Interprétation

□ $P((0,1) \mid (0,0), \text{droite}) = 0.8$

□ $P((1,0) \mid (0,0), \text{droite}) = 0.1$

□ $P((0,0) \mid (0,0), \text{droite}) = 0.1$

L'action ne garantit pas le résultat.

MDP : Récompense (Reward)

Récompense (Reward)

- Après chaque transition d'état, l'agent reçoit une **récompense** qui évalue la qualité de l'action effectuée. Cette récompense est généralement notée :

$$R(s, a)$$

- L'objectif de l'agent est de choisir ses actions de manière à maximiser la somme cumulée des récompenses au cours du temps.

MDP : Politique(Policy)

- Une **politique** définit le comportement de l'agent.
- Elle spécifie quelle action doit être choisie dans chaque état.
 - Une politique est généralement notée :

$$\pi(a | s)$$

- Elle peut être **déterministe** (toujours la même action pour un état donné) ou **stochastique** (distribution de probabilité sur les actions).
- **(Optionnel)** γ : facteur de **discount** ($0 \leq \gamma \leq 1$) pour les récompenses futures.
Permet de prioriser les gains immédiats par rapport aux gains futurs

(MDP): Facteur de Discount

- ❑ Quand un agent prend des décisions, il reçoit des **récompenses** au fil du temps :
 - ❑ certaines tout de suite (immédiates)
 - ❑ d'autres plus tard (futures)
- ❑ Mais en pratique, une récompense reçue **maintenant** vaut souvent plus qu'une récompense reçue **dans 10 étapes**.
- ❑ Le **facteur de discount γ** permet donc de dire :
 - ❑ *"À quel point je me soucie du futur ?"*

◆ Valeur du facteur de discount

Le facteur de discount est compris entre :

$$0 \leq \gamma \leq 1$$

✓ Cas 1 : $\gamma = 0$

- L'agent ne se soucie **que du présent**
- Il devient **myope**
- Il cherche uniquement les récompenses immédiates
→ comportement **court terme**

✓ Cas 2 : γ proche de 1 (ex : 0.9, 0.99)

- L'agent prend en compte le **futur**
- Il peut accepter une petite récompense maintenant pour en avoir une plus grande plus tard
→ comportement **long terme**

◆ Exemple concret

Supposons que l'agent reçoit :

- 10 points **maintenant**
- 10 points **dans 2 étapes**

Si :

- $\gamma = 1 \rightarrow$ la récompense future vaut toujours 10
- $\gamma = 0.5 \rightarrow$ la récompense dans 2 étapes vaut :

$$10 \times (0.5)^2 = 2.5$$

Donc plus la récompense est **loin dans le temps**, plus elle est **réduite** (discounted).

MDP : Facteur de discount

- ❑ Le facteur de discount permet :
 - ❑ **d'éviter que la somme des récompenses futures soit infinie**
 - ❑ **de contrôler la vision court terme / long terme de l'agent**
 - ❑ **de stabiliser l'apprentissage dans des algorithmes comme :**
 - ❑ Q-Learning
 - ❑ Value Iteration
 - ❑ Deep Q-Network (DQN)

Processus de Décision de Markov (MDP)

Propriété de Markov

□ La propriété fondamentale d'un MDP est la **propriété de Markov**, selon laquelle la **probabilité de transition vers le prochain état dépend uniquement de l'état actuel et de l'action choisie, et non de l'historique complet des états précédents.**

□ Formellement :

$$P(s_{t+1}' | s_t' a_t' s_{t-1} a_{t-1} \dots) = P(s_{t+1}' | s_t a_t)$$

□ Cela signifie que l'état courant contient toute l'information pertinente nécessaire à la prise de décision.

Processus de Décision de Markov (MDP)

Objectif

Le but de l'apprentissage par renforcement est de trouver une politique optimale qui maximise la récompense cumulative attendue à long terme, souvent appelée retour (return).

■ Propriétés clés

- **Propriété de Markov :**

L'avenir ne dépend que de l'état courant et de l'action choisie, pas de l'historique complet.

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$$

- **Politique (π) :**

Une politique est une stratégie de décision qui indique quelle action prendre dans chaque état :

$$\pi(s) = a$$

- **Valeur d'un état ($V^\pi(s)$) :**

Valeur attendue des récompenses futures en suivant la politique π depuis l'état s :

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \mid s_0 = s, \pi \right]$$

- **Valeur d'action ($Q^\pi(s, a)$) :**

Valeur attendue en prenant l'action a dans l'état s et en suivant ensuite la politique π :

$$Q^\pi(s, a) = \mathbb{E} \left[R(s, a, s') + \gamma V^\pi(s') \right]$$

Équation de Bellman : son rôle dans les MDP

- L'équation de Bellman repose sur une idée très simple :
La valeur d'une décision aujourd'hui dépend du gain immédiat et de la qualité des décisions futures.
- Autrement dit, pour savoir si une action est bonne, on regarde :
 - ce qu'elle rapporte tout de suite
 - dans quel état futur elle nous mène
 - et si cet état futur est "prometteur"

The diagram shows the Bellman equation $V(s) = \max_a (R(s, a) + \gamma V(s'))$ with several annotations and colored boxes:

- $V(s)$** (blue box): The expected return (value) at the current state s is:
- $\max_a$** (light blue box): The maximum value of any possible action a for:
- $R(s, a)$** (purple box): The expected reward for taking action a at state s ...
- $+$** (pink box): ...plus the discount factor (gamma) multiplied by the value of the next state
- $\gamma V(s')$** (pink box):

Équation de Bellman : Pourquoi est-elle importante

- ❑ Dans un **Markov Decision Process (MDP)**, l'agent doit prendre une suite de décisions dans un environnement incertain.
- ❑ Sans l'équation de Bellman :
 - ❑ il faudrait analyser **toutes les trajectoires possibles**
 - ❑ **le problème deviendrait exponentiellement complexe**
 - ❑ **le calcul serait pratiquement impossible**
- ❑ L'équation de Bellman rend le problème **calculable** en le décomposant étape par étape

Équation de Bellman : Pourquoi est-elle importante

Elle joue trois rôles fondamentaux :

1. Évaluer une stratégie

- ☐ Elle permet de mesurer la qualité d'une politique (une façon d'agir).

2. Améliorer la stratégie

- ☐ Elle indique comment choisir une meilleure action en comparant les valeurs futures.

3. Base de tous les algorithmes

Les méthodes comme :

- ☐ Value Iteration
- ☐ Policy Iteration
- ☐ Q-learning
- ☐ Deep Q Networks

sont toutes basées sur ce principe.

Équation de Bellman : Intuition simple (exemple)

Imagine un robot qui doit atteindre une sortie :

- ❑ **Aller à gauche donne +1 immédiatement**
- ❑ **Aller à droite donne 0 maintenant, mais mène vers une grande récompense plus loin**

L'équation de Bellman permet de comprendre que **parfois une petite récompense immédiate n'est pas optimale si le futur est meilleur ailleurs.**

Exemple MDP

Supposons un robot dans une grille 2x2 :

États	Actions possibles
(0,0)	Haut, Bas, Gauche, Droite
(0,1)	Haut, Bas, Gauche, Droite
...	...

Etats= $\{(0,0),(0,1),(1,0),(1,1)\}$

❑ **Récompense : +1 si le robot atteint (1,1), 0 sinon.**

Facteur de discount : $\gamma = 0.9$

Le robot utilisera **Value Iteration** pour calculer $V(s)$ et déterminer la politique optimale pour atteindre (1,1) avec le maximum de récompense attendue.

Github link :

Exemple MDP

On a ici **4 états** représentant les positions dans la grille :
L'état **(1,1)** est le **but** que l'agent doit atteindre.

État	Position
(0,0)	Haut-gauche
(0,1)	Haut-droite
(1,0)	Bas-gauche
(1,1)	Bas-droite (état terminal)

Actions (actions)

- ❑ À chaque état, l'agent peut :
- ❑ monter (up)
- ❑ descendre (down)
- ❑ aller à gauche (left)
- ❑ aller à droite (right)

Facteur d'actualisation (gamma = 0.9)

Il mesure l'importance du futur :

- ❑ $\gamma \in [0,1]$
- ❑ **proche de 0** → l'agent pense au court terme
- ❑ **proche de 1** → l'agent pense au long terme
- ❑ **Ici 0.9 signifie que l'agent valorise fortement les récompenses futures**

Introduction au Deep Learning appliqué au Reinforcement Learning (DRL)

DRL : Deep Learning dans le Reinforcement Learning

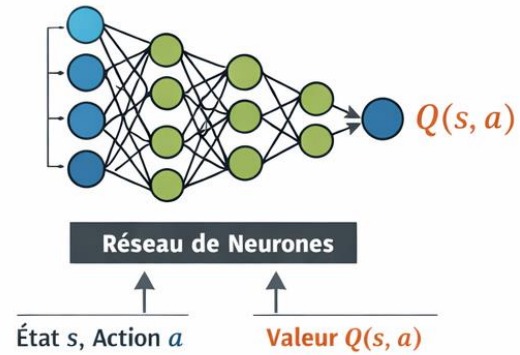
- Le **Deep Learning** est une branche de l'intelligence artificielle qui repose sur l'utilisation de **réseaux de neurones profonds** (Deep Neural Networks, DNN) **pour apprendre des représentations hiérarchiques de données**.
- Dans le contexte du **Reinforcement Learning (RL)**, le Deep Learning apporte une capacité clé : **approximer des fonctions complexes dans des espaces d'états et d'actions très grands ou continus, ce que les méthodes tabulaires classiques ne peuvent pas gérer**.

Deep Learning appliqué au Reinforcement Learning

Approximation de la Fonction de Valeur

$$Q(s, a) \approx Q_{\theta}(s, a)$$

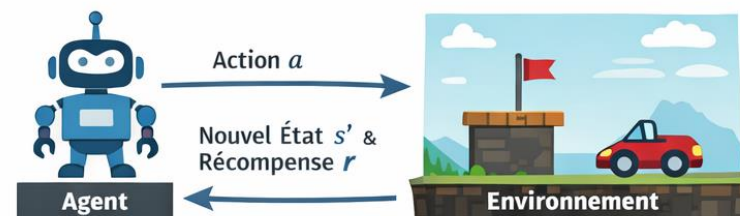
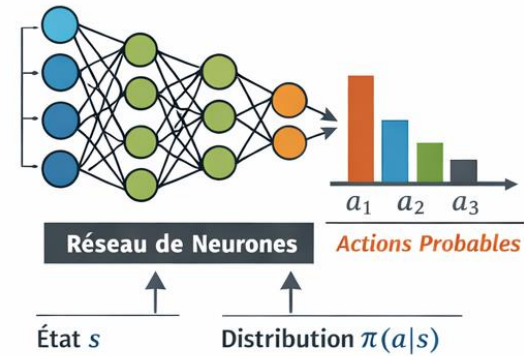
Réseau de Neurones



Approximation de la Politique

$$\pi(a|s) \approx \pi_{\theta}(a|s)$$

Réseau de Neurones



DRL : Approximation de la fonction de valeur

- Dans le RL classique, on définit une fonction de valeur $V(s)$ ou $Q(s, a)$ qui mesure, respectivement, **la qualité d'un état** ou **la qualité d'une action dans un état donné** :

$$Q(s, a) = \mathbb{E}[G_t' \mid s_t = s \ a_t = a]$$

- où G_t est la **récompense cumulative future**.
- Lorsque le nombre d'états est immense (par exemple, lorsqu'un état est représenté par une image ou un vecteur complexe de capteurs), il est **impossible de stocker tous les $Q(s, a)$ dans une table**. Le **Deep Learning** intervient ici en utilisant un réseau neuronal $Q_\theta(s, a)$ paramétré par θ pour approximer cette fonction.

DRL : Approximation de la fonction de valeur

- ❑ **Entrée du réseau** : vecteur représentant l'état s (ou une séquence d'états pour les tâches partielles observables).
- ❑ **Sortie du réseau** : valeurs $Q(s, a)$ pour chaque action possible a .
- ❑ Cette approche permet au réseau **d'interpoler et généraliser** la valeur d'actions dans des états non rencontrés pendant l'apprentissage.

DRL : Approximation de la politique

- ❑ Au lieu d'apprendre la fonction de valeur, certaines méthodes de RL cherchent à **apprendre directement la politique** $\pi(a | s)$, c'est-à-dire la probabilité de choisir une action a dans un état s .
- ❑ Un réseau neuronal peut être utilisé pour représenter la politique : $\pi_{\theta}(a | s)$, où les paramètres θ sont appris via l'optimisation de la récompense cumulée.
- ❑ Cette approche est particulièrement adaptée aux **espaces d'action continus**, où il est difficile de tabuler toutes les actions possibles.
- ❑ **Exemple :**
 - ❑ Pour un robot qui contrôle ses moteurs avec des valeurs continues, la sortie du réseau peut être un vecteur de moyennes et de variances d'une distribution normale sur les actions, permettant un échantillonnage stochastique des mouvements.

DRL: Avantages du Deep Learning pour le RL

- ❑ **Généralisation** : le réseau neuronal peut apprendre à prédire des valeurs ou des actions pour des états jamais rencontrés pendant l'entraînement.
- ❑ **Flexibilité des entrées** : les réseaux peuvent traiter des entrées complexes (images, vidéos, données multisensorielles).
- ❑ **Approximation non linéaire** : les fonctions de valeur et les politiques peuvent être hautement non linéaires, et les réseaux profonds sont capables de capturer ces complexités.
- ❑ **Extensibilité** : Deep RL peut évoluer vers des environnements très grands ou continus, ce qui est impossible avec un RL tabulaire classique

DRL : Limites et défis

Instabilité de l'apprentissage : Les réseaux neuronaux peuvent diverger si les mises à jour de la politique ou de la fonction de valeur sont trop agressives.

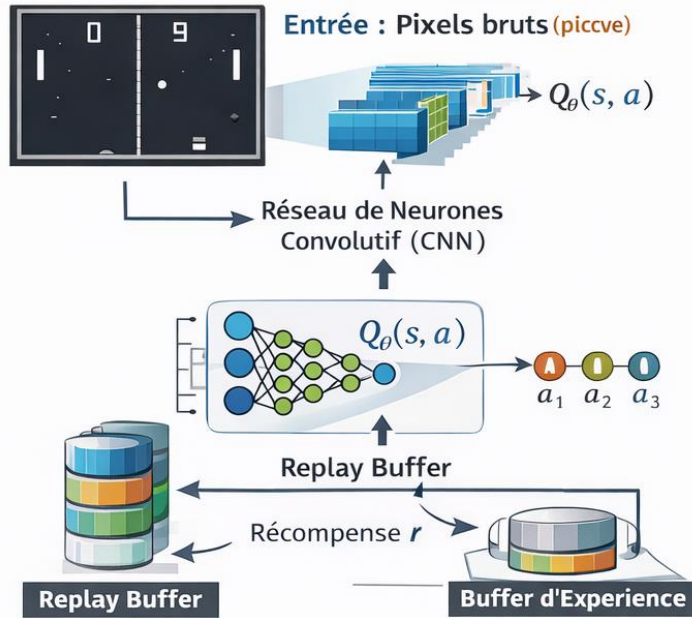
Besoins en données : l'apprentissage nécessite souvent des millions d'interactions avec l'environnement, ce qui peut être coûteux.

Sensibilité aux hyperparamètres : le choix de la taille du réseau, du taux d'apprentissage ou des fonctions d'activation influence fortement la performance.

Etude de cas

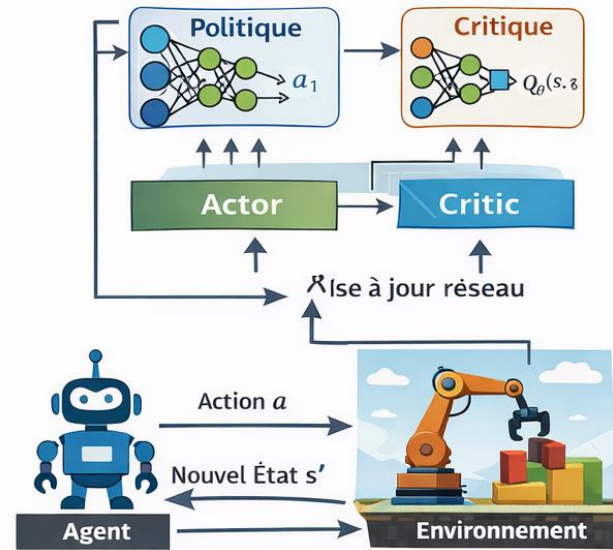
Étude de cas : RL dans les jeux vidéo et la robotique

RL dans les Jeux Vidéo (Deep Q-Network)



RL en Robotique

Entrée : États continus (angles, vitesses, capteurs)



	Jeux Vidéo	Robotique
Espace d'état	Discret (pixels)	Continu (angles, capteurs)
Espace d'action	Discret (gauche/droite, etc.)	Continu (moteurs)
Nombre d'épisodes	Elevé	Limité (si réel)
Technique RL typique	CNN, DQN	Actor-Critic (PPO, SAC)

Apprentissage par renforcement (Reinforcement Learning) : DQN (Deep Q-Network)

Introduction

- ❑ Le **Deep Q-Learning** ou **Deep Q Network (DQN)** est une extension de l'algorithme de base **Q-Learning**, qui utilise des réseaux de neurones profonds pour approximer les valeurs Q.
- ❑ Le Q-Learning traditionnel fonctionne bien pour des environnements ayant un nombre petit et fini d'états, mais **il rencontre des difficultés avec des espaces d'états grands ou continus en raison de la taille de la table Q**.
- ❑ Le **Deep Q-Learning** surmonte cette limitation en remplaçant la table Q par un réseau de neurones capable d'approximer les valeurs Q pour chaque paire état-action

Q-learning

❑ Le **Q-Learning traditionnel** est un algorithme d'apprentissage par renforcement (reinforcement learning) basé sur les valeurs, qui permet à un agent d'apprendre une politique optimale en interagissant avec un environnement.

❑ **Principe de base:**

❑ L'agent apprend une **fonction Q** (fonction d'action-valeur) qui estime la valeur d'une paire (**état, action**) :

$$Q(a, s)$$

❑ Cette valeur représente la récompense cumulative attendue si l'agent prend l'action a dans l'état s puis suit la meilleure politique possible par la suite.

❑ représente la qualité d'une action a dans un état s .

❑ Autrement dit : À quel point est-il bon de faire l'action a quand on est dans l'état s ?

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

où :

- α est le taux d'apprentissage
- γ est le facteur d'actualisation (discount factor)
- r est la récompense reçue
- s' est l'état suivant
- $\max_{a'} Q(s', a')$ est la meilleure valeur Q possible dans l'état suivant

Q-learning : Règle de mise à jour (équation de Bellman)

Q-learning : Caractéristiques principales

- ❑ Utilise une **table Q** pour stocker toutes les **valeurs état-action**
- ❑ Fonctionne bien pour des espaces d'états **petits et discrets**
- ❑ Algorithme **off-policy** (il apprend la politique optimale indépendamment de la stratégie d'exploration utilisée)
- ❑ Souvent combiné avec une stratégie **ϵ -greedy** pour équilibrer exploration et exploitation

Stratégie ϵ -greedy (epsilon-greedy)

- ❑ La stratégie **ϵ -greedy** est une méthode utilisée en **apprentissage par renforcement (Reinforcement Learning)** pour résoudre le problème suivant :
 - ❑ **Explorer de nouvelles actions**
 - ❑ ou **Exploiter les connaissances actuelles pour maximiser la récompense**
- ❑ On appelle cela le **dilemme exploration / exploitation**.

ϵ -greedy : Principe

- ❑ À chaque étape, l'agent doit choisir une action :
- ❑ Avec une probabilité ϵ (epsilon) → il explore
 - ❑ Il choisit une action au hasard
- ❑ Avec une probabilité $1 - \epsilon$ → il exploite
- ❑ Il choisit l'action qui a la plus grande Q-value

Formellement :

- ❑ Soit :
 - ❑ $\epsilon \in [0,1]$
- ❑ Alors :
 - ❑ Si un nombre aléatoire $< \epsilon$ → action aléatoire (exploration)
 - ❑ Sinon → meilleure action selon le modèle (exploitation)

Exemple:

- ❑ epsilon = 0.2

Alors :

- ❑ 20% du temps → l'agent essaie une action au hasard
- ❑ 80% du temps → il choisit la meilleure action connue

Q-learning ---> DQL

- ❑ Dans le , on utilise une **table Q** pour stocker les valeurs :

$$Q(\text{état}, \text{action})$$

- ❑ **Mais lorsque :**

- ❑ les états sont trop nombreux (ex : images, vidéos, positions continues)

- ❑ ou l'environnement est complexe

- ❑ Alors il devient impossible d'utiliser une table.

- ❑ Le **DQN remplace cette table par un réseau de neurones profond** qui approxime la fonction :

$$Q(s, a)$$

Donc :

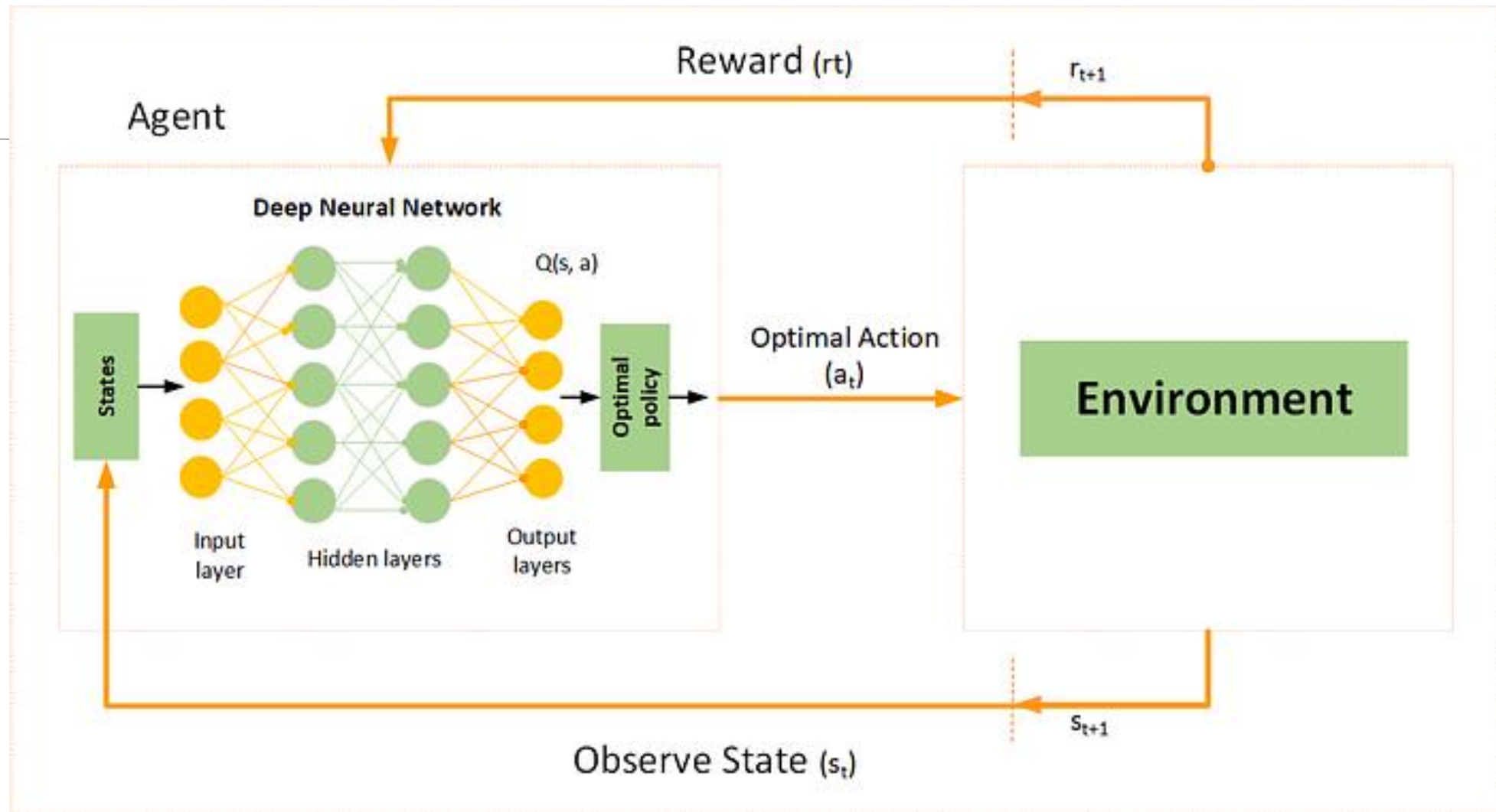
- ❑ **Entrée** : l'état s (par exemple une image du jeu)

- ❑ **Sortie** : les valeurs Q pour toutes les actions possibles

DQN

Architecture

- ❑ **L'architecture DQN (Réseau Q Profond)** est une méthode **d'apprentissage par renforcement** qui combine :
 - ❑ le **Q-learning** (apprentissage de la meilleure action à choisir)
 - ❑ avec les **réseaux de neurones profonds** (deep learning)
- ❑ **Elle permet à un agent (ex : un robot, un joueur virtuel) d'apprendre à prendre de bonnes décisions dans un environnement complexe**



DQN : Concepts clés du Deep Q-Learning

- ❑ **Approximation de la fonction Q** : Au lieu d'utiliser une table pour stocker les valeurs Q pour chaque paire état-action, le DQN utilise un réseau de neurones pour approximer les valeurs Q. **L'entrée du réseau est l'état, et la sortie est un ensemble de valeurs Q pour toutes les actions possibles.**
- ❑ **Rejeu d'expérience (Experience Replay)** : **Pour stabiliser l'entraînement, le DQN utilise un tampon de mémoire (replay buffer) pour stocker les expériences (état, action, récompense, état suivant).** Le réseau est entraîné sur des mini-lots aléatoires d'expériences provenant de ce tampon, ce qui rompt la corrélation entre les expériences consécutives et améliore l'efficacité de l'échantillonnage.

DQN : Concepts clés du Deep Q-Learning

- ❑ **Réseau cible (Target Network):** Le DQN introduit un second réseau de neurones, appelé **réseau cible**, qui est utilisé pour calculer les valeurs Q cibles. Ce réseau cible est mis à jour moins fréquemment que le réseau principal afin d'éviter des oscillations rapides pendant l'apprentissage.
- ❑ **Équation de Bellman dans le DQN :** La règle de mise à jour du DQN est basée sur l'équation de Bellman, de manière similaire au Q-Learning :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)$$

DQN : Concepts clés du Deep Q-Learning

- ❑ θ sont les poids du réseau Q principal,
- ❑ θ^- sont les poids du réseau Q cible,
- ❑ s est l'état courant,
- ❑ a est l'action effectuée,
- ❑ r est la récompense reçue,
- ❑ s' est l'état suivant,
- ❑ $\max_{a'} Q(s', a')$ est la valeur Q maximale pour l'état suivant.

Applications réelles du Deep Q-Learning

Le DQN et ses variantes ont été appliqués à de nombreuses tâches complexes, notamment :

1. IA pour les jeux vidéo

Le DQN a gagné en popularité grâce à son application dans les jeux Atari, où l'agent a appris à atteindre des performances surhumaines en interagissant avec l'environnement.

2. Robotique

Dans les tâches de manipulation robotique, le DQN peut être utilisé pour aider les robots à apprendre des comportements complexes, tels que saisir et placer des objets ou naviguer dans des environnements dynamiques.

3. Véhicules autonomes

Les voitures autonomes peuvent utiliser le DQN pour apprendre des politiques de conduite dans des environnements simulés, leur permettant de naviguer dans le trafic, d'éviter des obstacles et d'optimiser les itinéraires.

Applications réelles du Deep Q-Learning

3. Véhicules autonomes

Les voitures autonomes peuvent utiliser le DQN pour apprendre des politiques de conduite dans des environnements simulés, leur permettant de naviguer dans le trafic, d'éviter des obstacles et d'optimiser les itinéraires.

4. Finance

Le DQN peut être appliqué aux stratégies de trading algorithmique, où un agent apprend à maximiser le profit en prenant des **décisions d'achat/vente/conservation en temps réel**.

5. Santé

Dans le domaine de la santé, le DQN peut être utilisé pour personnaliser les plans de traitement en apprenant à partir des données des patients et en maximisant les résultats de santé à long terme.

Applications réelles du Deep Q-Learning

- ❑ Le DQN étend le Q-Learning traditionnel en exploitant des réseaux de neurones pour approximer les valeurs Q dans des environnements ayant des espaces d'états grands ou continus.
- ❑ Grâce à des techniques comme le rejeu d'expérience et l'utilisation d'un réseau cible, le DQN surmonte de nombreuses limitations du Q-Learning de base.
- ❑ Sa capacité à apprendre des tâches complexes à partir d'entrées de grande dimension en fait un outil puissant pour résoudre des problèmes du monde réel tels que les jeux, la robotique et la conduite autonome.

Episode vs Epoque

❑ Épisode (en apprentissage par renforcement)

- ❑ Un épisode correspond à **une partie complète de l'environnement**, du **début jusqu'à la fin** (ou **jusqu'à ce que l'agent atteigne un état terminal**).
- ❑ Dans **CartPole**, un épisode commence avec le poteau droit et se termine lorsque le poteau tombe ou que le nombre maximum de pas est atteint.
- ❑ **Pendant un épisode, l'agent collecte des expériences** (état, action, récompense, nouvel état) et apprend à partir de celles-ci.

Exemple :

- ❑ Épisode 1 → le poteau tombe au pas 50 → récompense totale = 50
- ❑ Épisode 2 → le poteau reste droit pendant 120 pas → récompense totale = 120
- ❑ Donc **un épisode est une séquence complète d'actions jusqu'à la fin de la tâche**.

❑ Époque (Epoch en apprentissage supervisé / Deep Learning)

- ❑ Une époque correspond à **un passage complet sur l'ensemble du jeu de données d'entraînement**.
- ❑ En apprentissage supervisé, on dispose d'un dataset (X, Y) fixe et une époque signifie que le modèle a **vu toutes les données une fois** pour mettre à jour ses poids.

Exemple :

- ❑ Dataset = 10 000 images → 1 époque = le modèle a vu toutes les 10 000 images
- ❑ On peut entraîner sur 50 époques → le modèle voit le dataset 50 fois en tout

CartPole

- ❑ CartPole est un environnement classique de contrôle dans le domaine de l'apprentissage par renforcement (Reinforcement Learning), souvent considéré comme le « **Hello World** » de cette discipline.
- ❑ Il simule un pendule inversé où un bâton (le pôle) est fixé à un chariot qui se déplace le long d'un rail sans friction.
- ❑ Vidéo : <https://youtu.be/2u1REHeHMrG>

Objectif et Mécanisme

- ❑ **Le but est d'équilibrer le pôle à la verticale en déplaçant le chariot vers la gauche ou la droite.**
- ❑ **Action :** Le système accepte deux actions discrètes : **pousser le chariot vers la gauche (0)** ou **vers la droite (1) avec une force fixe.**
- ❑ **Récompense :** Pour chaque étape où le pôle reste debout, l'agent reçoit une récompense de +1.
- ❑ **Conditions d'échec :** Un épisode se termine si l'angle du pôle dépasse environ $\pm 12^\circ$ (ou 15° selon la version), si le chariot sort des limites de la piste (± 2.4 unités), ou si une limite de temps (souvent 500 étapes pour la version v1) est atteinte.

Simulations

- ❑ <https://github.com/hk027/Lacite/tree/main/DRL/MarkovDecisionProcess>
- ❑ <https://github.com/hk027/Lacite/tree/main/DRL/DQL>

Apprentissage par renforcement (Reinforcement Learning) : DQN (Deep Q-Network)

Introduction

- ❑ Le **Deep Q-Learning** ou **Deep Q Network (DQN)** est une extension de l'algorithme de base **Q-Learning**, qui utilise des réseaux de neurones profonds pour approximer les valeurs Q.
- ❑ Le Q-Learning traditionnel fonctionne bien pour des environnements ayant un nombre petit et fini d'états, mais **il rencontre des difficultés avec des espaces d'états grands ou continus en raison de la taille de la table Q**.
- ❑ Le **Deep Q-Learning** surmonte cette limitation en remplaçant la table Q par un réseau de neurones capable d'approximer les valeurs Q pour chaque paire état-action

Q-learning

❑ Le **Q-Learning traditionnel** est un algorithme d'apprentissage par renforcement (reinforcement learning) basé sur les valeurs, qui permet à un agent d'apprendre une politique optimale en interagissant avec un environnement.

❑ **Principe de base:**

❑ L'agent apprend une **fonction Q** (fonction d'action-valeur) qui estime la valeur d'une paire (**état, action**) :

$$Q(a, s)$$

❑ Cette valeur représente la récompense cumulative attendue si l'agent prend l'action a dans l'état s puis suit la meilleure politique possible par la suite.

❑ représente la qualité d'une action a dans un état s .

❑ Autrement dit : À quel point est-il bon de faire l'action a quand on est dans l'état s ?

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

où :

- α est le taux d'apprentissage
- γ est le facteur d'actualisation (discount factor)
- r est la récompense reçue
- s' est l'état suivant
- $\max_{a'} Q(s', a')$ est la meilleure valeur Q possible dans l'état suivant

Q-learning : Règle de mise à jour (équation de Bellman)

Q-learning : Caractéristiques principales

- ❑ Utilise une **table Q** pour stocker toutes les **valeurs état-action**
- ❑ Fonctionne bien pour des espaces d'états **petits et discrets**
- ❑ Algorithme **off-policy** (il apprend la politique optimale indépendamment de la stratégie d'exploration utilisée)
- ❑ Souvent combiné avec une stratégie **ϵ -greedy** pour équilibrer exploration et exploitation

Stratégie ϵ -greedy (epsilon-greedy)

- ❑ La stratégie **ϵ -greedy** est une méthode utilisée en **apprentissage par renforcement (Reinforcement Learning)** pour résoudre le problème suivant :
 - ❑ **Explorer de nouvelles actions**
 - ❑ ou **Exploiter les connaissances actuelles pour maximiser la récompense**
- ❑ On appelle cela le **dilemme exploration / exploitation**.

ϵ -greedy : Principe

- ❑ À chaque étape, l'agent doit choisir une action :
- ❑ Avec une probabilité ϵ (epsilon) → il explore
 - ❑ Il choisit une action au hasard
- ❑ Avec une probabilité $1 - \epsilon$ → il exploite
- ❑ Il choisit l'action qui a la plus grande Q-value

Formellement :

- ❑ Soit :
 - ❑ $\epsilon \in [0,1]$
- ❑ Alors :
 - ❑ Si un nombre aléatoire $< \epsilon$ → action aléatoire (exploration)
 - ❑ Sinon → meilleure action selon le modèle (exploitation)

Exemple:

- ❑ epsilon = 0.2

Alors :

- ❑ 20% du temps → l'agent essaie une action au hasard
- ❑ 80% du temps → il choisit la meilleure action connue

Q-learning ---> DQL

- ❑ Dans le Q-learning , on utilise une **table Q** pour stocker les valeurs :
 $Q(\text{état}, \text{action})$

- ❑ **Mais lorsque :**

- ❑ les états sont trop nombreux (ex : images, vidéos, positions continues)
 - ❑ ou l'environnement est complexe

- ❑ Alors il devient impossible d'utiliser une table.

- ❑ Le **DQN remplace cette table par un réseau de neurones profond** qui approxime la fonction :

$$Q(s, a)$$

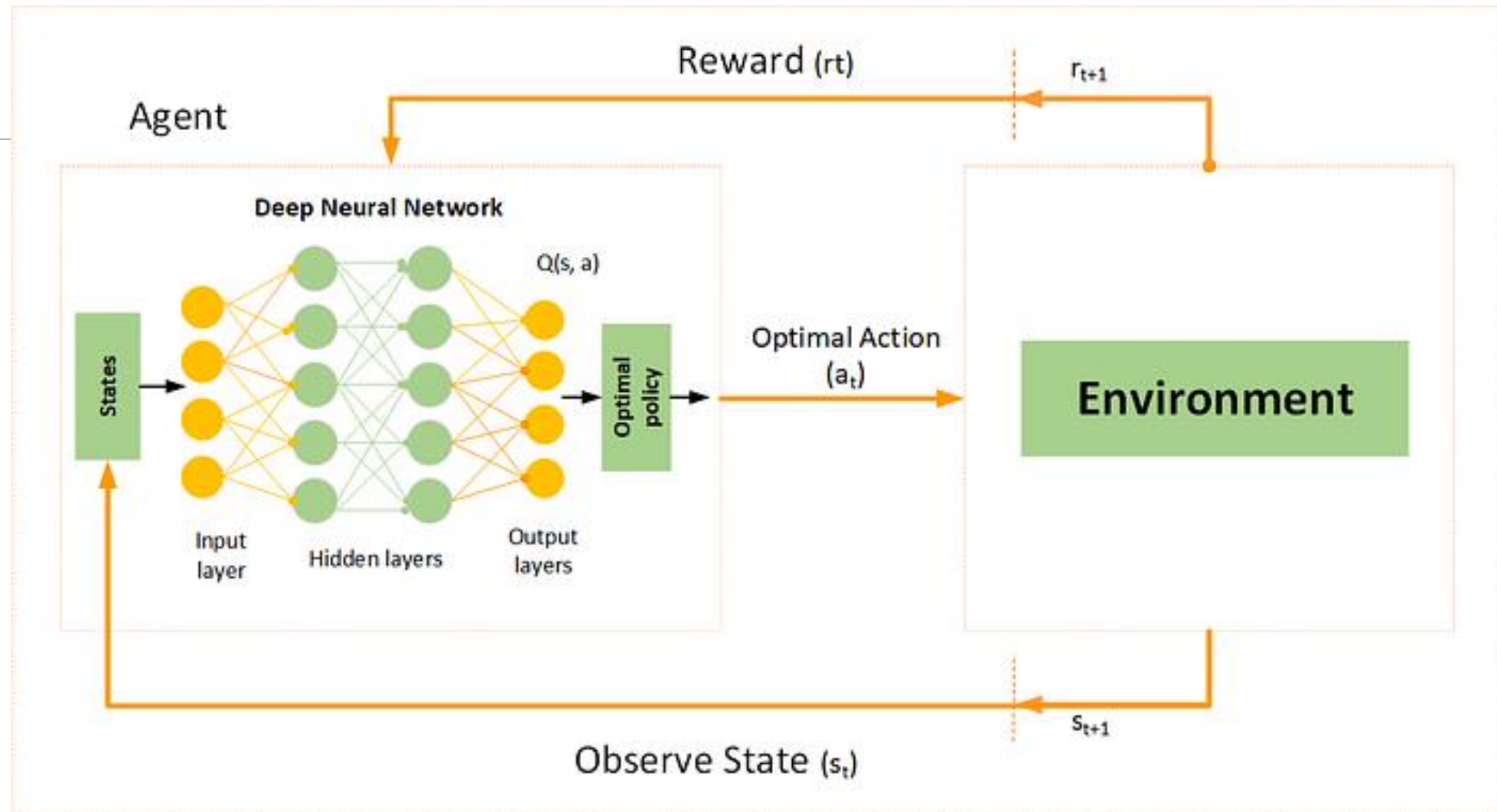
Donc :

- ❑ **Entrée** : l'état s (par exemple une image du jeu)
 - ❑ **Sortie** : les valeurs Q pour toutes les actions possibles

DQN

Architecture

- ❑ **L'architecture DQN (Réseau Q Profond)** est une méthode **d'apprentissage par renforcement** qui combine :
 - ❑ le **Q-learning** (apprentissage de la meilleure action à choisir)
 - ❑ avec les **réseaux de neurones profonds** (deep learning)
- ❑ **Elle permet à un agent (ex : un robot, un joueur virtuel) d'apprendre à prendre de bonnes décisions dans un environnement complexe**



DQN : Concepts clés du Deep Q-Learning

- ❑ **Approximation de la fonction Q** : Au lieu d'utiliser une table pour stocker les valeurs Q pour chaque paire état-action, **le DQN utilise un réseau de neurones pour approximer les valeurs Q. L'entrée du réseau est l'état, et la sortie est un ensemble de valeurs Q pour toutes les actions possibles.**
- ❑ **Rejeu d'expérience (Experience Replay)** : **Pour stabiliser l'entraînement, le DQN utilise un tampon de mémoire (replay buffer) pour stocker les expériences (état, action, récompense, état suivant).** Le réseau est entraîné sur des mini-lots aléatoires d'expériences provenant de ce tampon, ce qui rompt la corrélation entre les expériences consécutives et améliore l'efficacité de l'échantillonnage.

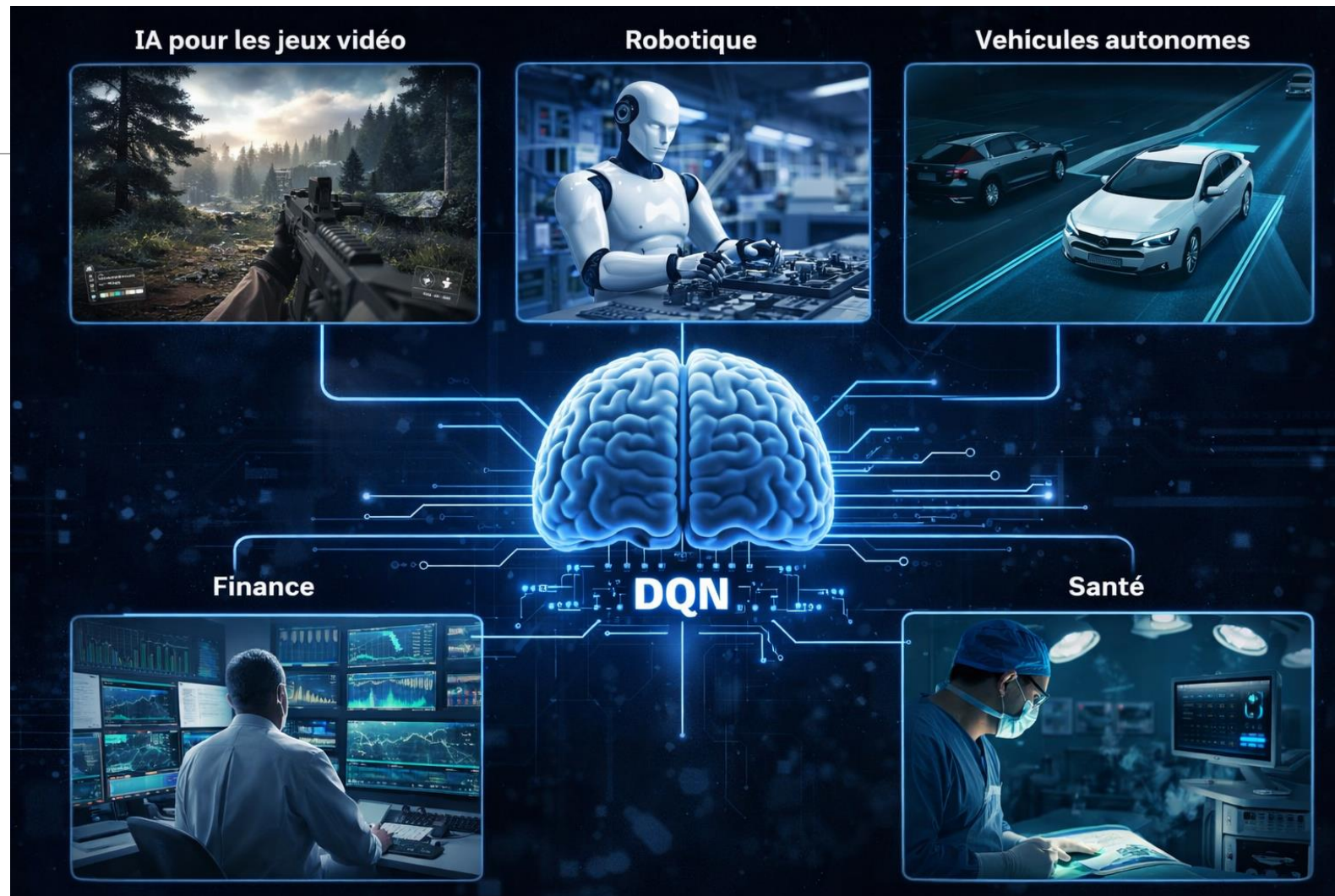
DQN : Concepts clés du Deep Q-Learning

- ❑ **Réseau cible (Target Network):** Le DQN introduit un second réseau de neurones, appelé **réseau cible**, qui est utilisé pour calculer les valeurs Q cibles. Ce réseau cible est mis à jour moins fréquemment que le réseau principal afin d'éviter des oscillations rapides pendant l'apprentissage.
- ❑ **Équation de Bellman dans le DQN :** La règle de mise à jour du DQN est basée sur l'équation de Bellman, de manière similaire au Q-Learning :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)$$

DQN : Concepts clés du Deep Q-Learning

- θ sont les poids du réseau Q principal,
- θ^- sont les poids du réseau Q cible,
- s est l'état courant,
- a est l'action effectuée,
- r est la récompense reçue,
- s' est l'état suivant,
- $\max_{a'} Q(s', a')$ est la valeur Q maximale pour l'état suivant.



Applications réelles du Deep Q-Learning

Le DQN et ses variantes ont été appliqués à de nombreuses tâches complexes, notamment :

1. IA pour les jeux vidéo

Le DQN a gagné en popularité grâce à son application dans les jeux Atari, où l'agent a appris à atteindre des performances surhumaines en interagissant avec l'environnement.

2. Robotique

Dans les tâches de manipulation robotique, le DQN peut être utilisé pour aider les robots à apprendre des comportements complexes, tels que saisir et placer des objets ou naviguer dans des environnements dynamiques.

3. Véhicules autonomes

Les voitures autonomes peuvent utiliser le DQN pour apprendre des politiques de conduite dans des environnements simulés, leur permettant de naviguer dans le trafic, d'éviter des obstacles et d'optimiser les itinéraires.

Applications réelles du Deep Q-Learning

4. Finance

Le DQN peut être appliqué aux stratégies de trading algorithmique, où un agent apprend à maximiser le profit en prenant des **décisions d'achat/vente/conservation en temps réel**.

5. Santé

Dans le domaine de la santé, le DQN peut être utilisé pour personnaliser les plans de traitement en apprenant à partir des données des patients et en maximisant les résultats de santé à long terme.

Applications réelles du Deep Q-Learning

- ❑ Le **DQN** étend le **Q-Learning** traditionnel en exploitant des réseaux de neurones pour approximer les valeurs **Q** dans des environnements ayant des espaces d'états grands ou continus.
- ❑ Grâce à des techniques comme le rejeu d'expérience et l'utilisation d'un réseau cible, le DQN surmonte de nombreuses limitations du Q-Learning de base.
- ❑ Sa capacité à apprendre des tâches complexes à partir d'entrées de grande dimension en fait un outil puissant pour résoudre des problèmes du monde réel tels que les jeux, la robotique et la conduite autonome.

Episode vs Epoque

❑ Épisode (en apprentissage par renforcement)

- ❑ Un épisode correspond à **une partie complète de l'environnement**, du **début jusqu'à la fin** (ou **jusqu'à ce que l'agent atteigne un état terminal**).
- ❑ Dans **CartPole**, un épisode commence avec le poteau droit et se termine lorsque le poteau tombe ou que le nombre maximum de pas est atteint.
- ❑ **Pendant un épisode, l'agent collecte des expériences** (état, action, récompense, nouvel état) et apprend à partir de celles-ci.

Exemple :

- ❑ Épisode 1 → le poteau tombe au pas 50 → récompense totale = 50
- ❑ Épisode 2 → le poteau reste droit pendant 120 pas → récompense totale = 120
- ❑ Donc **un épisode est une séquence complète d'actions jusqu'à la fin de la tâche**.

❑ Époque (Epoch en apprentissage supervisé / Deep Learning)

- ❑ Une époque correspond à **un passage complet sur l'ensemble du jeu de données d'entraînement**.
- ❑ En apprentissage supervisé, on dispose d'un dataset (X, Y) fixe et une époque signifie que le modèle a **vu toutes les données une fois** pour mettre à jour ses poids.

Exemple :

- ❑ Dataset = 10 000 images → 1 époque = le modèle a vu toutes les 10 000 images
- ❑ On peut entraîner sur 50 époques → le modèle voit le dataset 50 fois en tout

CartPole

- ❑ **CartPole** est un environnement classique de contrôle dans le domaine de l'apprentissage par renforcement (Reinforcement Learning), souvent considéré comme le « **Hello World** » de cette discipline.
- ❑ Il simule un pendule inversé où un bâton (le pôle) est fixé à un chariot qui se déplace le long d'un rail sans friction.
- ❑ Vidéo : <https://youtu.be/2u1REHeHMrq>

Objectif et Mécanisme

- ❑ **Le but est d'équilibrer le pôle à la verticale en déplaçant le chariot vers la gauche ou la droite.**
- ❑ **Action :** Le système accepte deux actions discrètes : **pousser le chariot vers la gauche (0)** ou **vers la droite (1) avec une force fixe.**
- ❑ **Récompense :** Pour chaque étape où le pôle reste debout, l'agent reçoit une récompense de +1.
- ❑ **Conditions d'échec :** Un épisode se termine si l'angle du pôle dépasse environ $\pm 12^\circ$ (ou 15° selon la version), si le chariot sort des limites de la piste (± 2.4 unités), ou si une limite de temps (souvent 500 étapes pour la version v1) est atteinte.

Simulations

- ❑ <https://github.com/hk027/Lacite/tree/main/DRL/MarkovDecisionProcess>
- ❑ <https://github.com/hk027/Lacite/tree/main/DRL/DQL>

GANs (Generative Adversarial Networks)

GAN : Définitions

- ❑ Les réseaux antagonistes génératifs (GAN) sont une innovation récente et passionnante dans le domaine du machine Learning.
- ❑ Les GAN sont des modèles *génératifs*: ils créent de nouvelles instances de données qui ressemblent à vos données d'entraînement.
- ❑ Par exemple, les GAN peuvent créer des images qui ressemblent à des photographies de visages humains, même si ces visages n'appartiennent à aucune personne réelle.
- ❑ Ces images ont été créées par un GAN:



Figure 1: Images générées par un GAN créé par NVIDIA.

GAN : Définitions

- ❑ Les **GAN (Generative Adversarial Networks)** sont un modèle **d'apprentissage profond** proposé en 2014 par Ian Goodfellow.
- ❑ Un **GAN** est composé de **deux réseaux de neurones** qui **s'entraînent simultanément dans un processus compétitif** :
 - ❑ **Le générateur (Generator)** : crée (génère) de nouvelles données artificielles.
 - ❑ **Le discriminateur (Discriminator)** : essaie de distinguer les données réelles des données générées.
- ❑ **Le générateur tente de produire des données réalistes**, tandis que **le discriminateur tente de détecter les données fausses**.

Cette compétition améliore progressivement les performances des deux réseaux

GAN : Définitions

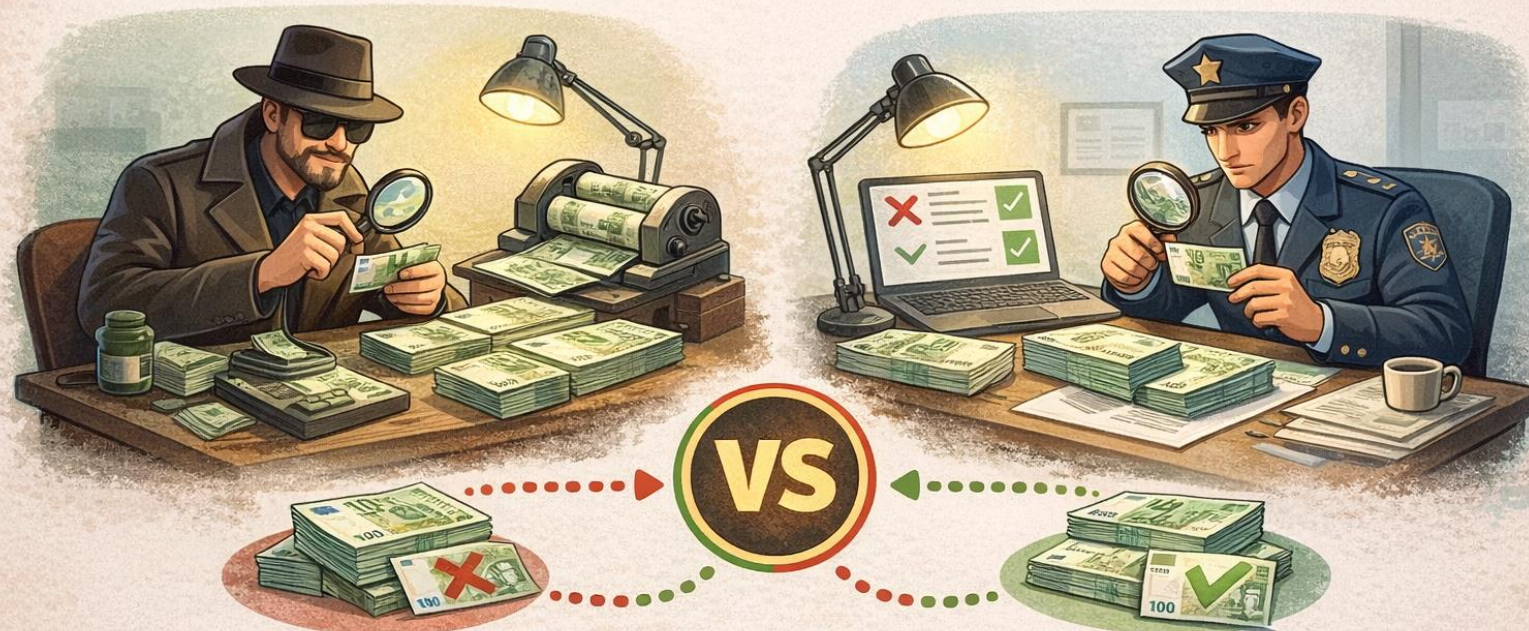
- ❑ **Un réseau antagoniste génératif (GAN)** se compose de deux parties:
 - ❑ Le **générateur** apprend à générer des données plausibles. Les instances générées deviennent des exemples d'entraînement négatifs pour le discriminateur.
 - ❑ Le **discriminateur** apprend à distinguer les données factices du générateur des données réelles. Le discriminateur pénalise le générateur pour la production de résultats invraisemblables.

Lorsque l'entraînement commence, le générateur produit des données manifestement fausses, et le discriminateur apprend rapidement à les identifier comme telles

Idée fondamentale : jeu à somme nulle

- ❑ Le fonctionnement des GAN repose sur un **jeu à somme nulle**, c'est-à-dire que :
 - ❑ **le gain d'un modèle correspond à la perte de l'autre.**
- ❑ **Dans ce système :**
 - ❑ **si le discriminateur détecte correctement les fausses données, il gagne**
 - ❑ **si le générateur réussit à tromper le discriminateur, il gagne**
- ❑ Cette compétition pousse les deux réseaux à s'améliorer progressivement.
- ❑ Une analogie classique est celle :
 - ❑ d'un **faussaire (le générateur)** qui fabrique de **faux billets**
 - ❑ d'un **inspecteur (le discriminateur)** qui doit **détecter les faux.**
- ❑ **Avec le temps, le faussaire devient de plus en plus habile et les faux billets deviennent très difficiles à distinguer des vrais.**

Une analogie classique :



Faussaire (Générateur)
fabrique de faux billets

Inspecteur (Discriminateur)
détecte les faux

Importance des GAN

Les GAN sont devenus très importants dans le domaine de Machine Learning et de Computer Vision pour plusieurs raisons :

Génération de données réalistes

Les GAN permettent de créer des données très proches des données réelles :

☐ **images**

☐ **vidéos**

☐ **audio**

☐ **texte**

Cela ouvre de nouvelles possibilités dans l'intelligence artificielle.

Importance des GAN

❑ **Résolution du problème du manque de données**

- ❑ Dans certains domaines (médecine, sécurité, recherche), les données sont rares. Les GAN permettent de **générer des données synthétiques** pour entraîner les modèles.

❑ **Amélioration des performances des modèles**

- ❑ Les données générées peuvent être utilisées pour :
 - ❑ **l'augmentation de données (data augmentation)**
 - ❑ améliorer les performances des systèmes d'apprentissage automatique.

❑ **Innovation dans les technologies créatives**

- ❑ Les GAN ont révolutionné la création d'images et d'effets visuels

Applications des GAN

APPLICATIONS

☐ Amélioration d'images (super-résolution)

- ☐ Applications :
- ☐ restauration de photos anciennes
- ☐ amélioration d'images satellites
- ☐ amélioration d'images médicales.

☐ Création artistique et design

- ☐ Les GAN peuvent générer :
- ☐ œuvres d'art
- ☐ textures pour jeux vidéo
- ☐ contenus visuels créatifs.
- ☐ Exemple : Runway ML

APPLICATIONS

Applications médicales

- ☐ Dans la médecine, les GAN sont utilisés pour :
- ☐ générer des images médicales synthétiques
- ☐ améliorer la qualité des IRM et scanners
- ☐ aider à entraîner des systèmes de **diagnostic assisté par IA**

Le Générateur (Generator)

- ❑ Le **générateur** *G* est un réseau de neurones chargé de **produire de nouvelles données artificielles**.
- ❑ **Entrée :**
 - ❑ un **vecteur aléatoire** appelé **bruit z**
- ❑ **Sortie :**
 - ❑ une **donnée synthétique** (image, texte, son, etc.).
- ❑ **Objectif :**
 - ❑ produire des données **aussi réalistes que possible** afin de tromper le **discriminateur**

Le Discriminateur (Discriminator)

- ❑ Le **discriminateur** D est un **réseau de neurones** qui agit comme un **classificateur**.
- ❑ **Entrée :**
 - ❑ une **donnée réelle ou générée**
- ❑ **Sortie :**
 - ❑ une **probabilité indiquant si la donnée est réelle ou fausse**.
- ❑ **Objectif :**
 - ❑ distinguer correctement :
 - ❑ les **vraies données** provenant du **dataset**
 - ❑ les **fausses données** créées par **le générateur**

Processus d'entraînement des GAN

L'entraînement d'un GAN se déroule de manière **itérative** :

1. Le **générateur** produit une **fausse donnée** à partir **d'un bruit aléatoire**.

2. Le discriminateur reçoit :

1. une **vraie donnée**

2. une **donnée générée**.

3. Le **discriminateur** apprend à **différencier les deux**.

4. Le **générateur** ajuste ses **paramètres** pour **tromper le discriminateur**.

Ce processus continue jusqu'à atteindre un **équilibre**, où les **données générées deviennent très proches des données réelles**



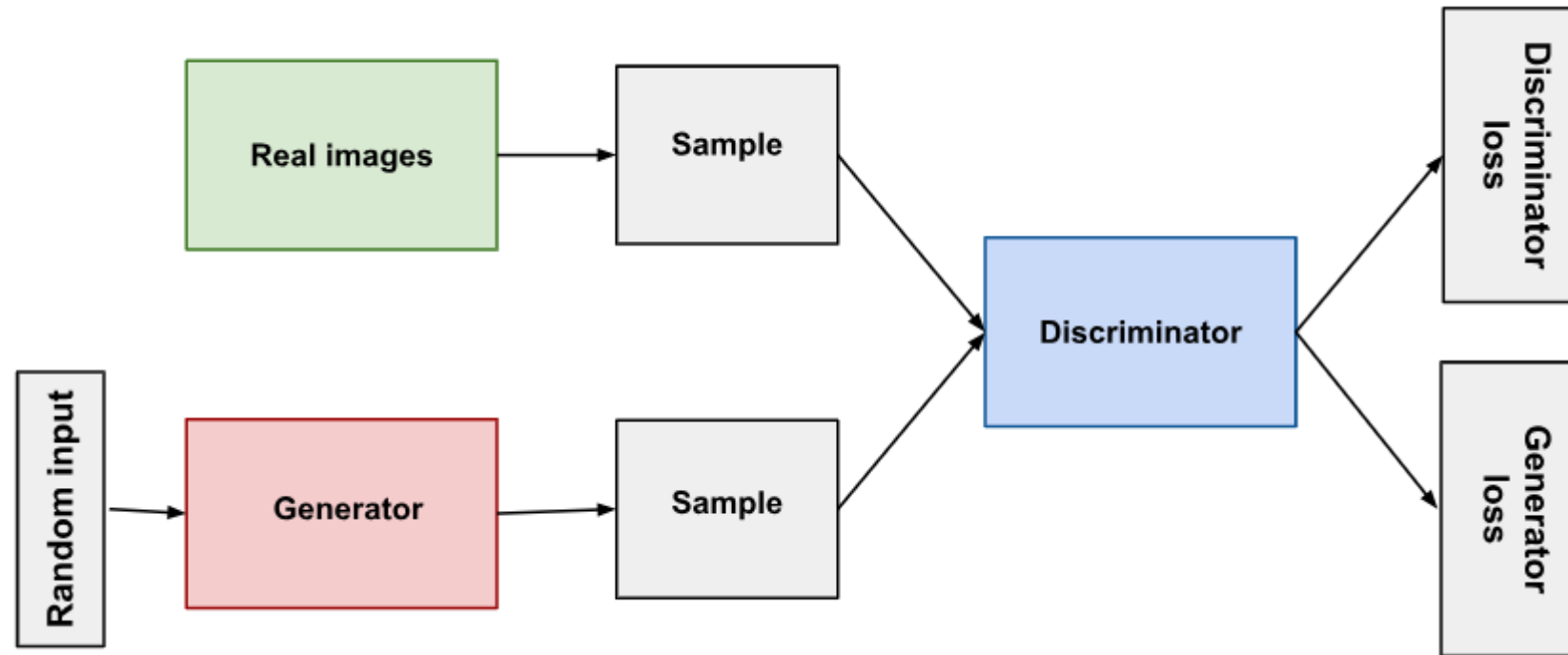
À mesure que l'entraînement progresse, le générateur se rapproche de la production d'une sortie capable d'induire en erreur le discriminateur:



Enfin, si l'entraînement du générateur se passe bien, le discriminateur a de plus en plus de mal à faire la différence entre le vrai et le faux. Il commence à classer les données factices comme réelles, et sa précision diminue.

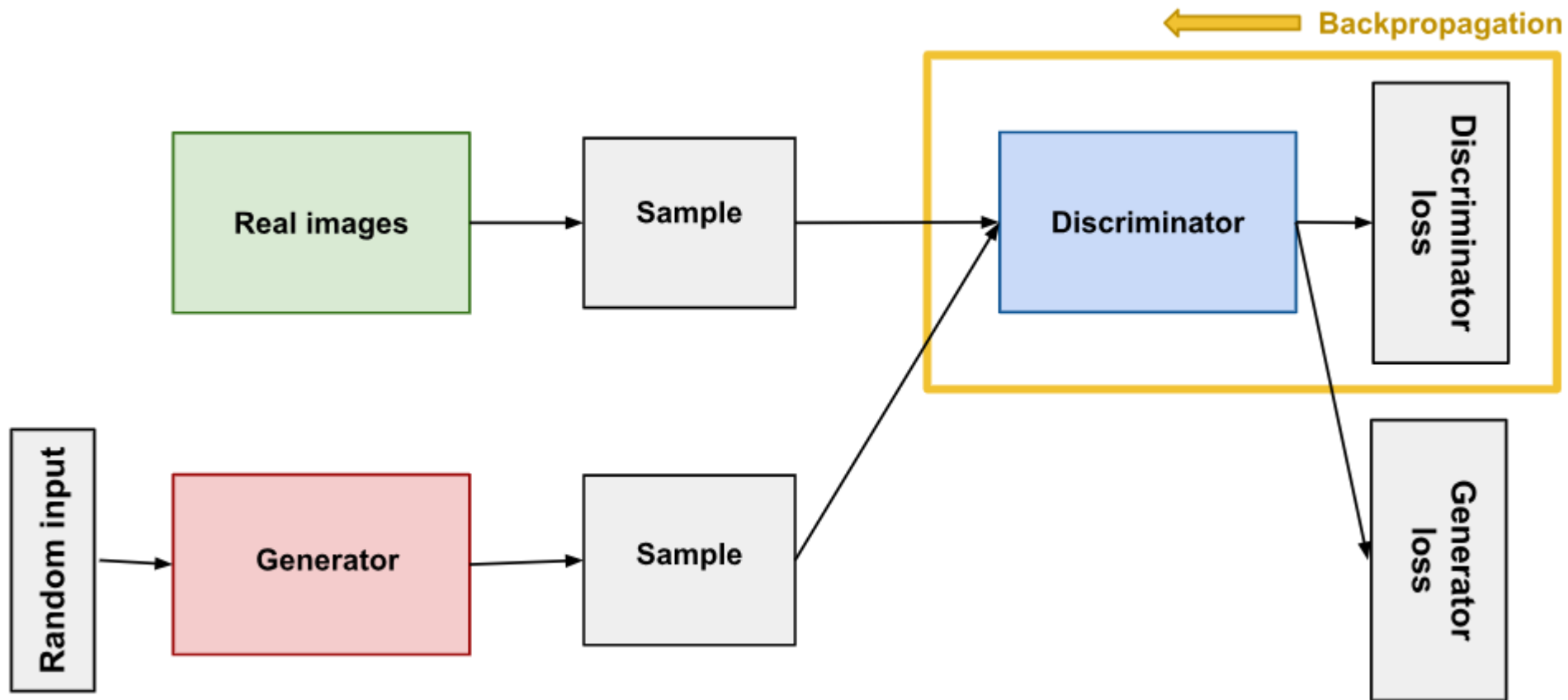


Architecture



- ❑ Le générateur et le discriminateur sont tous deux des réseaux de neurones.
- ❑ La sortie du générateur est connectée directement à l'entrée du discriminateur.
- ❑ Grâce à la propagation inverse, la classification du discriminateur fournit un signal que le générateur utilise pour mettre à jour ses poids.

Architecture : Discriminateur



Rétropropagation lors de l'entraînement du discriminateur.

Discriminateur : Données d'entraînement

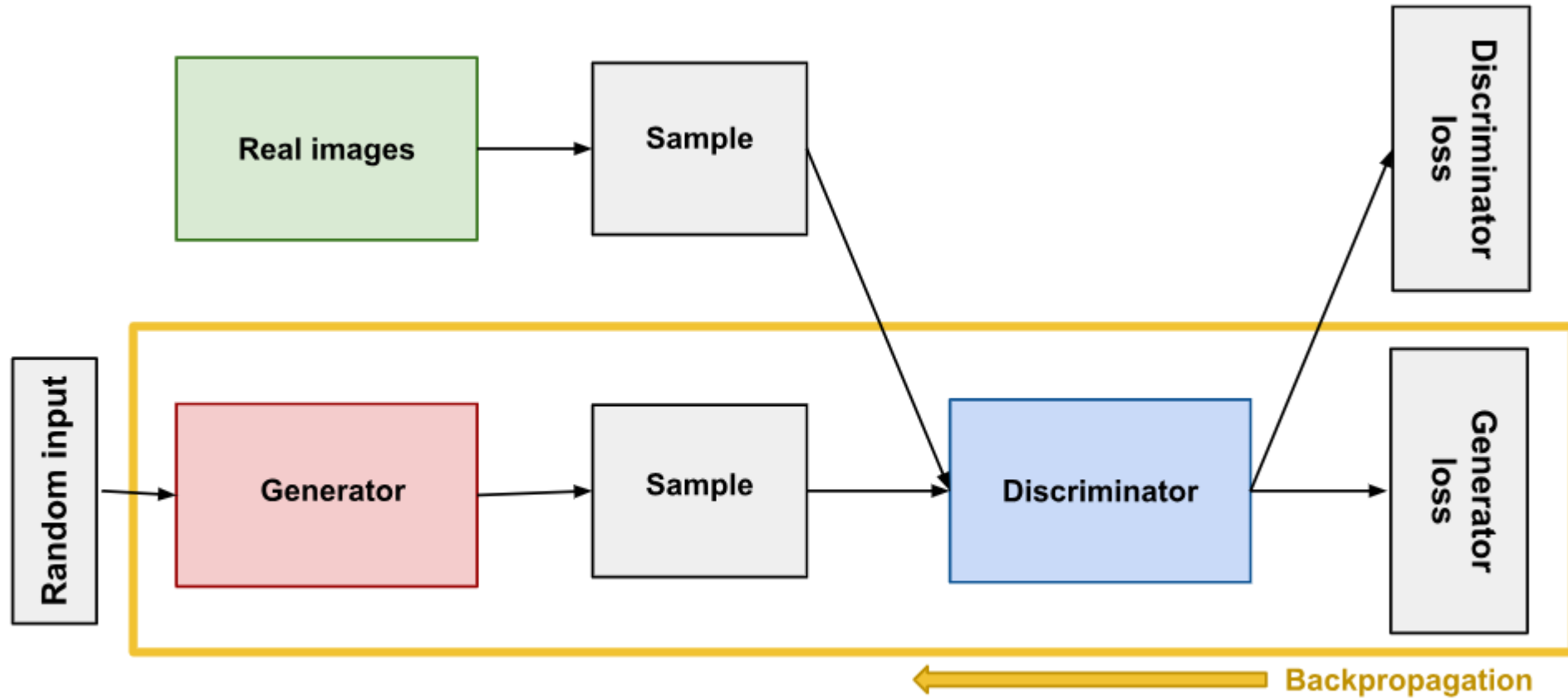
- ❑ Les données d'entraînement du discriminateur proviennent de deux sources:
 - ❑ Instances de données réelles, telles que des images réelles de personnes. Le discriminateur utilise ces instances comme exemples positifs pendant l'entraînement.
 - ❑ Instances de données factices créées par le générateur. Le discriminateur utilise ces instances comme exemples négatifs pendant l'entraînement.
- ❑ Dans la figure les deux cases "**Sample**" (**Exemple**) représentent ces deux sources de données qui alimentent le discriminateur.
- ❑ Lors de l'entraînement du discriminateur, le générateur ne s'entraîne pas. Ses poids restent constants pendant qu'il produit des exemples sur lesquels le discriminateur peut s'entraîner.

Discriminateur : Entraînement

- ❑ Le discriminateur se connecte à deux fonctions de [perte](#).
- ❑ Lors de **l'entraînement** du **discriminateur**, celui-ci ignore **la perte du générateur** et n'utilise que la **perte du discriminateur**.
- ❑ Nous utilisons la **perte du générateur** lors de **l'entraînement du générateur**.
- ❑ Pendant **l'entraînement** du **discriminateur**:
 - 1) Le **discriminateur** classe à la fois les **données réelles** et **les données factices** du **générateur**.
 - 2) La **perte du discriminateur** pénalise le discriminateur pour avoir mal classé une instance réelle comme fausse ou une instance fausse comme réelle.
 - 3) Le **discriminateur met à jour ses poids via la propagation inverse à partir de la perte du discriminateur via le réseau du discriminateur**

Le générateur

- ❑ La partie **générateur d'un GAN** apprend à créer de **fausses données** en **intégrant les commentaires du discriminateur**. Il apprend à faire en sorte que le discriminateur classe sa sortie comme réelle.
- ❑ L'**entraînement du générateur** nécessite une intégration plus étroite entre le **générateur** et le **discriminateur** que l'**entraînement** du **discriminateur**. La partie du GAN qui **entraîne le générateur** comprend les éléments suivants:
 - ❑ **entrée aléatoire**
 - ❑ **réseau générateur, qui transforme l'entrée aléatoire en instance de données**
 - ❑ **un réseau discriminateur, qui classe les données générées**
 - ❑ **sortie du discriminateur**
 - ❑ **perte du générateur, qui pénalise le générateur pour ne pas avoir réussi à tromper le discriminateur**



Rétropropagation lors de l'entraînement du générateur

Générateur : Saisie aléatoire - Bruit

- ❑ Les réseaux de neurones ont besoin d'une forme d'entrée. Normalement, nous saisissons des données avec lesquelles nous voulons effectuer une action, comme une instance que nous voulons classer ou sur laquelle nous voulons effectuer une prédiction.
- ❑ Mais que pouvons-nous utiliser comme entrée pour un réseau qui produit des instances de données entièrement nouvelles ?
- ❑ **Dans sa forme la plus élémentaire, un GAN utilise du bruit aléatoire comme entrée. Le générateur transforme ensuite ce bruit en sortie significative. En introduisant du bruit, nous pouvons amener le GAN à produire une grande variété de données, en échantillonnant à différents endroits de la distribution cible.**
- ❑ Les tests suggèrent que la distribution du bruit n'a pas beaucoup d'importance. Nous pouvons donc choisir quelque chose qui est facile à échantillonner, comme une distribution uniforme. Pour des raisons de commodité, l'espace à partir duquel le bruit est échantillonné est généralement de dimension inférieure à la dimensionnalité de l'espace de sortie

Générateur : discriminateur entraîne le générateur

- ❑ Pour entraîner un réseau de neurones, nous modifions ses poids afin de réduire l'erreur ou la perte de sa sortie.
- ❑ Dans notre GAN, cependant, le générateur n'est pas directement connecté à la perte que nous essayons d'affecter.
- ❑ Le générateur alimente le réseau discriminateur, et le *discriminateur* produit la sortie que nous essayons d'influencer. La perte du générateur pénalise le générateur pour avoir produit un échantillon que le réseau de discrimination classe comme faux.
- ❑ Ce morceau supplémentaire de réseau doit être inclus dans la rétropropagation. La rétropropagation ajuste chaque pondération dans la bonne direction en calculant son impact sur la sortie, c'est-à-dire comment la sortie changerait si vous modifiiez la pondération.
- ❑ Toutefois, l'impact d'un poids de générateur dépend de l'impact des poids de discriminants auxquels il est alimenté. La rétropropagation commence donc à la sortie et remonte à travers le discriminateur dans le générateur.

Générateur : discriminateur entraîne le générateur

□ En même temps, nous ne voulons pas que le discriminateur change pendant l'entraînement du générateur. Essayer de toucher une cible en mouvement rendrait un problème difficile encore plus difficile pour le générateur.

Nous entraînons donc le générateur avec la procédure suivante:

- a) Échantillonner un bruit aléatoire
- b) Générer la sortie du générateur à partir d'un bruit aléatoire échantillonné.
- c) Obtenez la classification "Réal" ou "Faux" du discriminateur pour la sortie du générateur.
- d) Calculez la perte à partir de la classification du discriminateur.
- e) Effectuez une rétropropagation à la fois dans le discriminateur et le générateur pour obtenir des gradients.
- f) Utilisez les gradients pour ne modifier que les poids du générateur.

□ Il s'agit d'une itération de l'entraînement du générateur.

Formation GAN

- ❑ Étant donné qu'un GAN contient deux réseaux entraînés séparément, son algorithme d'entraînement doit résoudre deux complications:
 - ❑ **Les GAN doivent jongler avec deux types d'entraînement différents (générateur et discriminateur).**
 - ❑ **La convergence des GAN est difficile à identifier.**
- ❑ **Entraînement alterné**
 - ❑ **Le générateur et le discriminateur ont des processus d'entraînement différents.**
 - ❑ L'entraînement du GAN se déroule en périodes alternées:
 1. **Le discriminateur s'entraîne pendant une ou plusieurs époques.**
 2. **Le générateur s'entraîne pendant une ou plusieurs époques.**
 3. Répétez les étapes 1 et 2 pour continuer à entraîner les réseaux du générateur et du discriminateur.

Formation GAN

- ❑ **Maintenir le générateur constant pendant la phase d'entraînement du discriminateur. Alors que l'entraînement du discriminateur tente de déterminer comment distinguer les données réelles des données factices, il doit apprendre à reconnaître les défauts du générateur. Il s'agit d'un problème différent pour un générateur bien entraîné que pour un générateur non entraîné qui produit une sortie aléatoire.**
- ❑ **De même, nous maintenons le discriminateur constant pendant la phase d'entraînement du générateur. Sinon, le générateur tenterait de toucher une cible en mouvement et pourrait ne jamais converger.**
- ❑ **C'est cet aller-retour qui permet aux GAN de résoudre des problèmes génératifs autrement insolubles.** Nous obtenons un point d'appui dans le problème génératif difficile en commençant par un problème de classification beaucoup plus simple. À l'inverse, si vous ne parvenez pas à entraîner un classificateur pour faire la différence entre les données réelles et celles générées, même pour la sortie initiale du générateur de nombres aléatoires, vous ne pouvez pas démarrer l'entraînement du GAN

Convergence

- ❑ À mesure que le générateur s'améliore avec l'entraînement, les performances du discriminateur se dégradent, car il ne peut pas facilement faire la différence entre le réel et le faux. Si le générateur réussit parfaitement, le discriminateur a une précision de 50 %. En fait, le discriminateur lance une pièce pour effectuer sa prédiction.
- ❑ Cette progression pose un problème de convergence du GAN dans son ensemble: les commentaires du discriminateur deviennent moins pertinents au fil du temps. Si le GAN continue de s'entraîner au-delà du point où le discriminateur fournit des commentaires complètement aléatoires, le générateur commence à s'entraîner sur des commentaires indésirables, et sa propre qualité peut s'effondrer.
- ❑ Pour un GAN, la convergence est souvent un état éphémère plutôt que stable

Fonction de perte globale

$$\min_G \max_D V(D,G) = E_x[\log D(x)] + E_z[\log(1 - D(G(z)))]$$

- $D(x)$ correspond à l'estimation du discriminateur de la probabilité que l'instance de données réelle x soit réelle.
- E_x est la valeur attendue pour toutes les instances de données réelles.
- $G(z)$ est la sortie du générateur lorsqu'il reçoit le bruit z .
- $D(G(z))$ est l'estimation du discriminateur de la probabilité qu'une instance factice soit réelle.
- E_z est la valeur attendue pour toutes les entrées aléatoires du générateur (en fait, la valeur attendue pour toutes les instances factices générées $G(z)$)

Codes

<https://github.com/hk027/Lacite/blob/main/GAN/gan.ipynb>

Apprentissage par
renforcement
(Reinforcement
Learning) :
DCGAN (Deep
Convolutional GAN

Rappel : GAN (Generative Adversarial Network)

- Un **GAN** est composé de deux réseaux :
 - **Générateur (G)** : crée des données (images, texte...)
 - **Discriminateur (D)** : distingue les vraies données des fausses
- Ils s'entraînent en compétition (**jeu min-max**)

GAN : Limites

❑ Les **GAN (Generative Adversarial Networks)** sont très puissants pour générer des données réalistes (**images, audio, texte**), mais ils présentent plusieurs **limites importantes** .

❑ **Voici quelques limites :**

- ❑ **Instabilité de l'entraînement**
- ❑ **Mode Collapse (effondrement de mode)**
- ❑ **Difficulté d'évaluation**
- ❑ **Sensibilité aux hyperparamètres**
- ❑ **Coût computationnel élevé**
- ❑ **Problèmes de convergence**
- ❑ **Manque de contrôle sur la génération**
- ❑ **Risques éthiques et abus**
- ❑ **Sensibilité aux données d'entraînement**
- ❑ **Difficulté à apprendre certaines distributions**

Instabilité de l'entraînement

- ❑ L'apprentissage repose sur un jeu compétitif entre le **générateur** et le **discriminateur**.
- ❑ Il est souvent difficile de trouver un équilibre :
 - ❑ **Si le discriminateur est trop fort** → **le générateur n'apprend pas**.
 - ❑ **Si le générateur est trop fort** → **le discriminateur devient inutile**.
- ❑ **Résultat : oscillations ou divergence pendant l'entraînement**

Mode Collapse (effondrement de mode)

- ❑ Le générateur produit peu de diversité.
- ❑ Il peut générer toujours les mêmes types d'images (ou très similaires).
- ❑ Cela réduit fortement la qualité globale du modèle

Difficulté d'évaluation

- ❑ Contrairement à d'autres modèles, il n'existe pas de métrique simple et universelle.
- ❑ Les mesures comme :
 - ❑ FID (Fréchet Inception Distance*) et IS (Inception Score) restent imparfaites.
- ❑ L'évaluation repose souvent sur une inspection humaine !!!

Sensibilité aux hyperparamètres

- ❑ **Les GAN nécessitent un réglage fin :**
 - ❑ **taux d'apprentissage**
 - ❑ **architecture réseau**
 - ❑ **fonctions de perte**
- ❑ **Un mauvais choix peut faire échouer complètement l'entraînement**

Coût computationnel élevé

- ❑ Entraîner deux réseaux simultanément (**générateur** + **discriminateur**) est :
 - ❑ **long**
 - ❑ **coûteux en GPU**
- ❑ Cela limite leur utilisation dans certains contextes

Problèmes de convergence

- ❑ Il n'y a aucune garantie de convergence.
- ❑ Le modèle peut :
 - ❑ ne jamais atteindre un équilibre
 - ❑ produire des résultats incohérents

Manque de contrôle sur la génération

- ❑ Difficulté à contrôler précisément ce qui est généré.
- ❑ Les GAN classiques ne permettent pas facilement :
 - ❑ d'imposer des contraintes spécifiques
 - ❑ d'interpréter les variables latentes

Risques éthiques et abus

- ❑ Utilisation possible pour :
 - ❑ **Deepfakes** (Un deepfake est un enregistrement vidéo ou audio réalisé ou modifié grâce à l'intelligence artificielle).
 - ❑ **manipulation d'images**
 - ❑ **désinformation**
- ❑ Cela pose des **problèmes de sécurité et d'éthique**.

Sensibilité aux données d'entraînement

- ❑ **Si les données sont biaisées ou insuffisantes :**
 - ❑ le GAN reproduit ces biais
 - ❑ **mauvaise généralisation**

Difficulté à apprendre certaines distributions

- ❑ **Les GAN ont du mal avec :**
 - ❑ données discrètes
 - ❑ structures complexes (ex : texte structuré)

Solutions aux limites de GAN

- ❑ Les **GAN** sont très puissants mais **fragiles** et **difficiles à maîtriser**.
- ❑ C'est pourquoi plusieurs variantes ont été proposées pour corriger ces limites, comme :
 - ❑ **DCGAN**
 - ❑ **WGAN (Wasserstein GAN)**
 - ❑ **StyleGAN**

DCGAN

DCGAN (Deep Convolutional GAN)

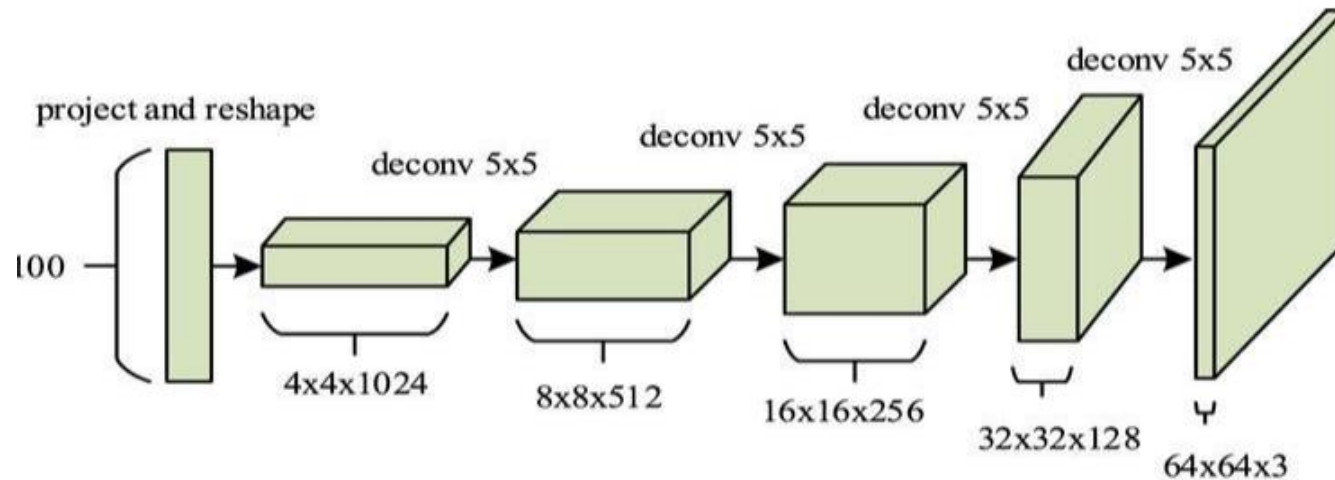
Définition

- ❑ Le **DCGAN (Deep Convolutional Generative Adversarial Network)** est une **variante des réseaux antagonistes génératifs (GAN)** qui repose entièrement sur des **réseaux de neurones convolutifs profonds (CNN)** pour améliorer la génération d'images.
- ❑ Introduit par Alec Radford et ses collaborateurs en 2015, DCGAN constitue une avancée majeure dans le domaine de l'apprentissage profond.

Principe

- ❑ Comme tout GAN, **DCCGAN** met en jeu deux réseaux :
 - ❑ **Le générateur (G)**
 - ❑ → Apprend à produire des images réalistes à partir d'un vecteur aléatoire (bruit)
 - ❑ **Le discriminateur (D)**
 - ❑ → Apprend à distinguer les vraies images des images générées
- ❑ L'apprentissage se fait de manière **adversariale** :
 - ❑ **le générateur s'améliore pour tromper le discriminateur, tandis que le discriminateur devient plus performant pour détecter les faux**

Architecture DCGAN : Générateur (G)



□ Générateur (G)

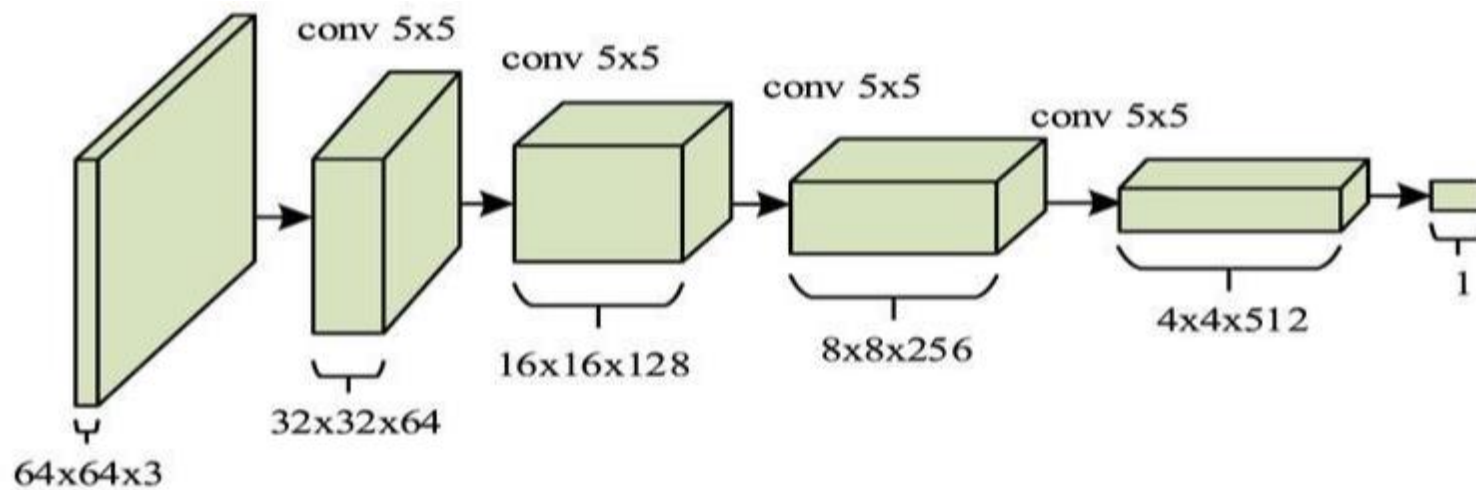
□ Utilise des **couches de convolution (deconvolution) transposée**

□ Transforme un **vecteur aléatoire en image**

□ Upsampling progressif

Code Générateur : <https://github.com/hk027/Lacite/blob/main/DCGAN/generateur.ipynb>

Architecture DCGAN : Discriminateur (D)



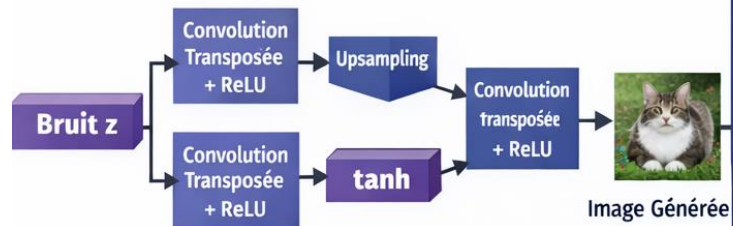
- ❑ **Utilise des convolutions classiques**
- ❑ **Downsampling de l'image**
- ❑ **Sortie : probabilité réel/faux**

Code Discriminateur : <https://github.com/hk027/Lacite/blob/main/DCGAN/discriminateur.ipynb>

Architecture du DCGAN

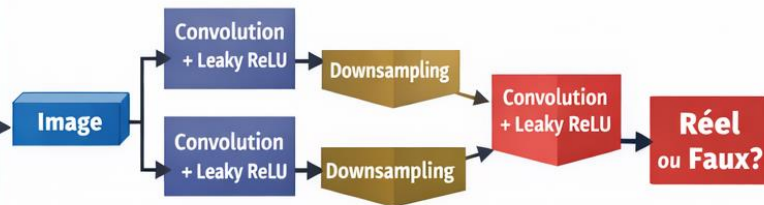
Générateur (G)

- Vecteur aléatoire (Bruit)
- Convolutions Transposées (Upsampling)
- Sortie : Image Générée



Discriminateur (D)

- Image Réelle ou Fausse
- Convolutions Classiques (Downsampling)
- Sortie : Probabilité Réel / Faux



Compétition Adversariale

Comparaison des fonctions

Fonction	Où ?	Avantages	Inconvénients
ReLU	Générateur (couches internes)	❑ Apprentissage rapide ❑ Évite le vanishing gradient ❑ Simple à calculer	❑ Peut créer des neurones morts (sortie 0 permanente)
tanh	Sortie du générateur	❑ Sorties normalisées (-1 à 1) ❑ Génère des données centrées ❑ Améliore la stabilité de sortie	❑ Vanishing gradient pour grandes valeurs ❑ Peut ralentir l'apprentissage
LeakyReLU	Discriminateur	❑ Évite les neurones morts ❑ Laisse passer un gradient même pour $x < 0$ ❑ Stabilise le discriminateur	❑ Un peu plus complexe à calculer ❑ Choix du coefficient α peut influencer la performance

Bonnes pratiques DCGAN

☐ **Pas de pooling → remplacé par convolutions avec stride :**

- ☐ Dans les réseaux classiques, le **pooling (max pooling ou average pooling)** est souvent utilisé pour réduire la taille spatiale des images tout en conservant l'information principale.
- ☐ On utilise des convolutions avec stride > 1 pour le downsampling (réduction) dans le discriminateur.
- ☐ On utilise des convolutions transposées avec stride > 1 pour l'upsampling (augmentation) dans le générateur.

☐ **Pas de couches fully connected profondes :**

- ☐ Réduit les paramètres, **préserve la structure spatiale**, stabilise l'entraînement

☐ **Batch Normalization (stabilise l'entraînement)**

☐ **Activation :**

- ☐ ReLU (Générateur)
- ☐ LeakyReLU (Discriminateur)

☐ **Sortie générateur**

Pas de couches fully connected profondes

❑ Réduction des paramètres

- ❑ Les couches fully connected explosent le nombre de poids → plus difficile à entraîner.

❑ Préservation de la structure spatiale

- ❑ Une image a une structure (pixels voisins liés).
- ❑ Les convolutions respectent cela, contrairement aux couches denses qui "aplatissent" tout.

❑ Stabilité de l'entraînement

- ❑ Moins de paramètres = moins de risque de divergence (très important pour les GAN)

Batch Normalization

- ❑ Technique de normalisation des activations pendant l'entraînement.
- ❑ **Importance :**
 - ❑ **Accélère la convergence**
 - ❑ Évite que le générateur ou le discriminateur domine trop vite
- ❑ **Dans les GAN :**
 - ❑ **Sans BatchNorm → entraînement instable (oscillations, mode collapse)**
 - ❑ **Avec BatchNorm → gradients plus stables**

Fonctions d'activation

❑ ReLU (dans le Générateur)

❑ Produit des valeurs positives

❑ Permet un **gradient fort** (évite le problème de gradient nul)

❑ Pourquoi dans le générateur ?

❑ Aide à créer des motifs nets

❑ Facilite la propagation du signal

❑ LeakyReLU (dans le Discriminateur)

❑ Formule : $f(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha x & \text{sinon} \end{cases}$

❑ ReLU peut "mourir" (neurones bloqués à 0)

❑ LeakyReLU laisse passer un petit gradient même si $x < 0$

❑ Résultat :

❑ Discriminateur plus robuste

❑ Meilleure rétropropagation

❑ Sortie du Générateur (souvent tanh)

❑ DCGAN utilise généralement tanh en sortie.

❑ Pourquoi ?

❑ Sortie dans l'intervalle $[-1, 1]$

❑ Correspond aux images normalisées

❑ Avantages :

❑ Stabilité numérique

❑ Images plus réalistes

Variante	Problème traité	Idée principale	Avantages	Limites restantes
DCGAN (Deep Convolutional GAN)	Instabilité, mauvaise qualité d'image	Utilise des réseaux convolutionnels (CNN)	- Images plus réalistes- Entraînement plus stable que GAN classique	- Toujours sensible aux hyperparamètres
WGAN (Wasserstein GAN)	Instabilité, non-convergence	Distance de Wasserstein au lieu de la divergence JS	- Entraînement plus stable- Perte interprétable	- Nécessite contrainte de Lipschitz (complexe)
WGAN-GP (Gradient Penalty)	Problème de clipping dans WGAN	Ajoute une pénalité sur le gradient	- Plus stable que WGAN- Meilleure convergence	- Coût computationnel plus élevé
CGAN (Conditional GAN)	Manque de contrôle	Conditionne la génération (labels, classes)	- Contrôle sur les données générées	- Dépend des labels disponibles
CycleGAN	Données non appariées	Apprentissage de traduction entre deux domaines	- Pas besoin de données alignées	- Peut introduire des artefacts
StyleGAN	Qualité et contrôle	Manipulation du style à différents niveaux	- Images très réalistes (visages)- Contrôle fin	- Très coûteux en calcul
BigGAN	Qualité à grande échelle	Entraînement sur gros datasets avec architecture optimisée	- Très haute qualité d'image	- Besoin de ressources énormes
LSGAN (Least Squares GAN)	Gradient faible	Fonction de perte basée sur moindres carrés	- Apprentissage plus stable	- Moins performant que WGAN
EBGAN (Energy-Based GAN)	Instabilité	Discriminateur vu comme fonction d'énergie	- Interprétation différente	- Peu utilisé en pratique

Avantages :

- ❑ **Avantages**

- ❑ **Génère des images plus réalistes**
 - ❑ **Apprentissage plus stable que GAN classique**
 - ❑ **Exploite la structure spatiale des images**

Code

<https://github.com/hk027/Lacite/blob/main/DCGAN/DCGAN.ipynb>

Modèle	Spécificité
DCGAN	CNN pour images
CGAN	Conditionnement
WGAN	Entraînement stable
CycleGAN	Traduction sans paires
StyleGAN	Images ultra réalistes

WGAN

WGAN (Wasserstein GAN)

Le **WGAN (Wasserstein GAN)** est une amélioration des GAN classiques introduite pour résoudre des problèmes majeurs comme l'instabilité de l'entraînement et le *mode collapse*.

Définition

- ❑ **WGAN** remplace la fonction de perte classique des GAN par une distance plus stable appelée :
- ❑ **Distance de Wasserstein (Earth Mover Distance)**
- ❑ **Intuition :**
 - ❑ Elle mesure **le coût minimal pour transformer une distribution en une autre**
 - ❑ Imagine déplacer de la terre pour transformer une forme en une autre → coût = distance.

Différence avec un GAN classique

❑ **GAN classique :**

- ❑ Utilise la divergence **JS (Jensen-Shannon)**
- ❑ Problème : gradients qui disparaissent → entraînement instable

❑ **WGAN :**

- ❑ Utilise la distance de Wasserstein
- ❑ Donne un **signal de gradient utile même quand les distributions sont très différentes**

Fonction de perte WGAN

❑ Le GAN standard :

$$\log(D(x)) + \log(1 - D(G(z)))$$

❑ Le WGAN devient :

$$\mathcal{L} = \mathbb{E}[D(x)] - \mathbb{E}[D(G(z))]$$

❑ $\mathbb{E}[D(x)] \rightarrow$ moyenne des scores des vraies images

❑ $\mathbb{E}[D(G(z))]$ $\rightarrow$ moyenne des scores des fausses images

❑ x Données **réelles** (images du dataset)

❑ z Bruit aléatoire (entrée du générateur)

❑ $G(z)$ Données **générées** (fausses images)

❑ $D(\cdot)$

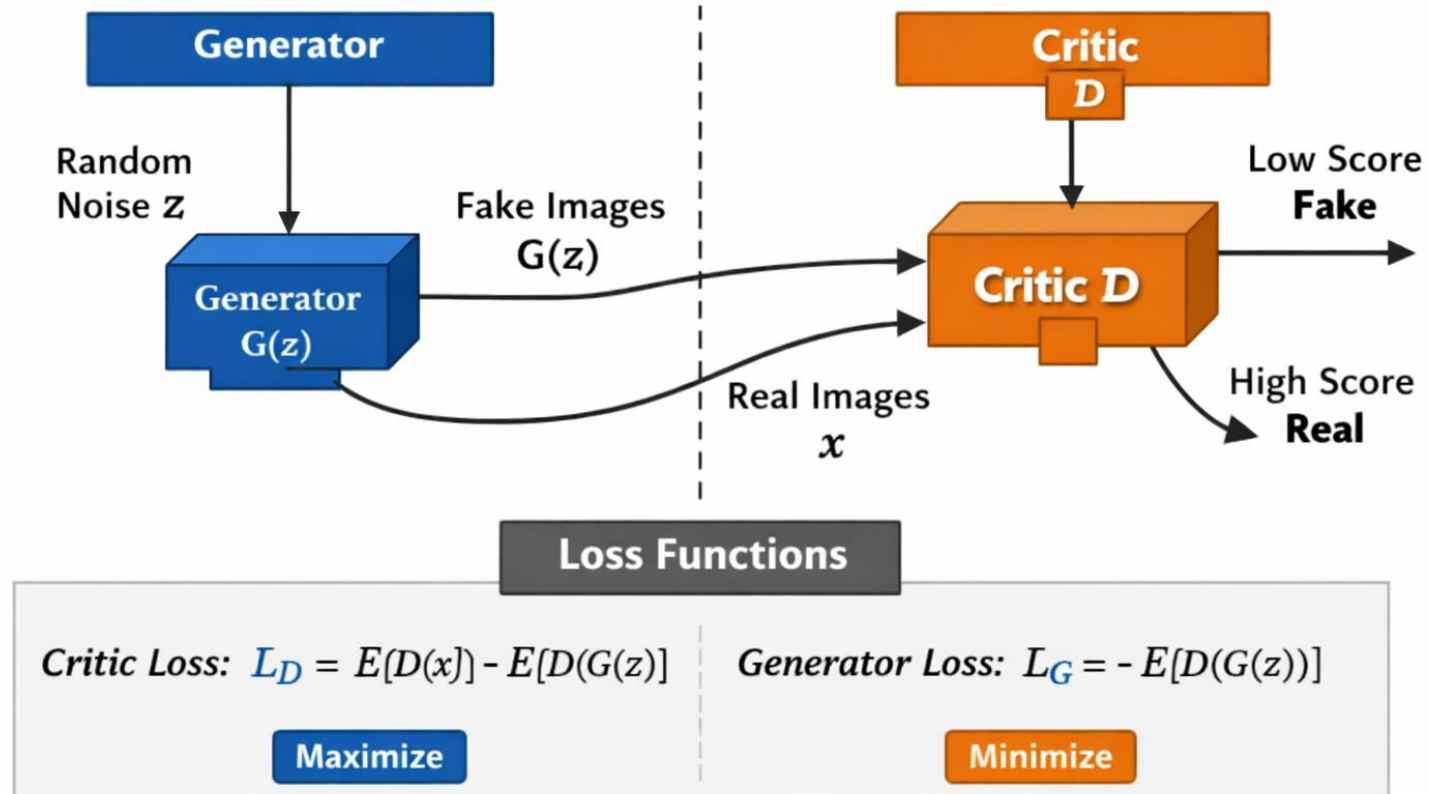
❑ Le **critic** (pas un discriminateur)

❑ Donne un **score réel** (pas une probabilité)

Architecture WGAN

- ❑ Très proche de **DCGAN**, mais avec changements :
- ❑ Pas de sigmoid à la sortie du critic
- ❑ Pas de BCE loss
- ❑ Optimisation avec RMSProp ou Adam modifié
- ❑ Critic entraîné plusieurs fois par itération (ex: 5 fois)

WGAN Architecture



Code

Lien : <https://github.com/hk027/Lacite/blob/main/DCGAN/WGAN/wgan.ipynb>